

Graduation Report

Presented at

The National School of Electronics and Telecommunications of Sfax

In order to obtain the

Engineering Diploma in: Data Engineering and Decisional System

Option :

Business Intelligence

By

Hammoudi Wissem

A modular Platform for Near-Real-Time Data Pipeline

Presented on 24/ 09/ 2025, before the review panel

Mrs.	Ben Massoud Ines	President
Mr.	BEN SAID Tarak	Examiner
Mr.	Koubaa Mohamed	Academic Supervisor
Mrs.	Benyakhlef Amal	Industrial Supervisor



Dedication

To my beloved mother, whose endless love, unwavering support, and countless sacrifices have made this journey possible. Your strength and wisdom have been my guiding light.

To my dear mother, Bassma..

To my father, who taught me the values of hard work, integrity, and perseverance. Your guidance and encouragement have shaped who I am today and inspired me to reach for my dreams.

To my dear father, Mohsen..

To my family and friends, who have supported me throughout this academic journey with their love, patience, and understanding.

To my industrial supervisors, **Nihed Mosbahi, Amal Ben Yakhlef, and Salma Gafsi,** whose guidance, encouragement, and valuable advice have greatly contributed to the completion of this work.

To all of you,

I dedicate this work.

Hammoudi Wissem



Table Of Contents

List Of Figures	vi
List Of Tables	vii
List of Abbreviations	viii
General Introduction	1
1 Preliminary Analysis	3
1.1 Introduction	4
1.2 General Presentation of the Organization	4
1.2.1 About Elyadata	4
1.2.2 Organizational Structure	5
1.2.3 Products and Services	5
1.3 Problem Statement	6
1.3.1 Current Data Processing Challenges	6
1.3.2 Limitations of Existing Solutions	6
1.3.3 Proposed Solution	7
1.4 Gantt Chart	8
1.5 Work Methodology	10
1.5.1 Scrum Framework Overview	10
1.5.2 Scrum Artifacts	11
1.5.3 Scrum Events	11
1.5.4 Scrum Roles	12
1.6 Version Control Strategy	12
1.7 Product Backlog	13
1.8 Conclusion	15
2 State of the Art	17
2.1 Introduction	18

2.2	Introduction to Modern Data Engineering	18
2.3	ETL vs. ELT: Core Data Processing Paradigms	19
2.3.1	Overview of Data Integration Strategies	19
2.3.2	Extract, Transform, Load (ETL) Methodology	19
2.3.3	Extract, Load, Transform (ELT) Methodology	21
2.3.4	Comparative Analysis and Decision Framework	22
2.3.5	Hybrid Approaches and Modern Implementations	22
2.4	Data Pipeline Scheduling and Orchestration	23
2.4.1	The Evolution from Batch to Event-Driven Processing	23
2.4.2	Core Concepts of Event-Driven Architecture	23
2.4.3	Implementation: Patterns and Technology Stack	25
2.4.4	Challenges and Future Directions	27
2.5	Conclusion	28
3	Project Scope And Conceptual Design	29
3.1	Introduction	30
3.2	System Requirements	30
3.2.1	Functional Requirements	30
3.2.2	Non-Functional Requirements	33
3.3	Global Use Case Diagram	34
3.4	Functional Modeling	35
3.4.1	Solution Overview	35
3.4.2	Class Diagram	36
3.4.3	C4 Models	37
3.4.4	Sequence Diagrams	40
3.5	Technology Stack and Work Environments	41
3.5.1	Development and Collaboration Tools	41
3.5.2	Backend Technology	41
3.5.3	Frontend Technology	43
3.5.4	Database and Storage	43
3.5.5	Containerization and Deployment	43
3.6	Conclusion	44
4	Data ingestion and transformation steps	45
4.1	Introduction	46
4.2	Sprint 1: Data Ingestion Step Implementation	46

TABLE OF CONTENTS

4.2.1	Sprint 1 Backlog	46
4.2.2	Sprint 1 Analysis	47
4.2.3	Sprint 1 Implementation	49
4.3	Sprint 2: Transformation Step Implementation	53
4.3.1	Sprint 2 Backlog	53
4.3.2	Sprint 2 Analysis	53
4.3.3	Sprint 2 Implementation	55
4.4	CONCLUSION	59
5	Advanced Transformation Types and Visualization Steps	60
5.1	Introduction	61
5.2	Sprint 3: Implementation of Multiple Transformation Types	61
5.2.1	Sprint 3 Backlog	61
5.2.2	Sprint 3 Analysis	62
5.2.3	Sprint 3 Implementation	65
5.3	Sprint 4: Visualization Step Implementation	71
5.3.1	Sprint 4 Backlog	71
5.3.2	Sprint 4 Analysis	71
5.3.3	Sprint 4 Implementation	73
5.4	Conclusion	76
6	Authentication, Authorization, and Change Data Capture	77
6.1	INTRODUCTION	78
6.2	Sprint 5: Authentication and Authorization Component	78
6.2.1	Sprint 5 Backlog	78
6.2.2	Sprint 5 Analysis	78
6.2.3	Sprint 5 Implementation	81
6.3	Sprint 6: Schema Change Detection Implementation	85
6.3.1	Sprint 6 Backlog	85
6.3.2	Sprint 6 Analysis	86
6.3.3	Sprint 6 Implementation	89
6.4	Conclusion	96
	GENERAL CONCLUSION	97
	Webography	97



List Of Figures

1.1	Elyadata's Logo	4
1.2	Project Timeline: Phase 1 (Months 1-3)	8
1.3	Project Timeline: Phase 2 (Months 4-6)	9
1.4	Scrum Framework Process Flow	10
1.5	The GitFlow Branching Model	13
2.1	Comparative Workflow Architecture of ETL and ELT Data Processing Pipelines	19
3.1	Global Use Case Diagram	35
3.2	Solution Overview	36
3.3	Class Diagram – System Architecture	36
3.4	System context diagram	38
3.5	Container diagram	39
3.6	Component diagram	40
3.7	Sequence Diagram for pipeline run	42
4.1	Data Ingestion Flow Diagram	47
4.2	Sequence Diagram: Creating a Pipeline with a Data Ingestion Step	48
4.3	dlt (data load tool) Logo	49
4.4	Airbyte Logo	49
4.5	Apache NiFi Logo	50
4.6	Meltano Architecture	50
4.7	Table and Column Selection Interface	52
4.8	Data Transformation Flow Diagram	54
4.9	Sequence Diagram: Creating a Pipeline with a Data Transformation Step	54
4.10	SQLMesh Logo	56
4.11	dbt Logo	56
4.12	Database Connection Configuration Interface	59
5.1	Join Transformation Workflow	62

5.2	Simple Transformation Workflow	63
5.3	Sequence Diagram: Creating Pipeline with Data Ingestion Step	64
5.4	Complete SQL Query Generation Model Comparison Metrics (Source [10]) . .	68
5.5	n8n Workflow Architecture for Agentic SQL Generation	70
5.6	Data Visualization Configuration Flow Diagram	72
5.7	Pipeline Creation Sequence Diagram	73
5.8	Apache Superset Logo	74
6.1	Authentication workflow with Keycloak	80
6.2	Authorization workflow with Admin and User roles	82
6.3	User workflow for schema change tracking and analysis	88
6.4	High-level data flow for hybrid schema and data change processing	90
6.5	Log-Based CDC: Reading changes directly from the database transaction log. .	91
6.6	Trigger-Based CDC: Using database triggers to populate a shadow audit table. .	92
6.7	Overview of the Debezium and Kafka architecture for change capture and processing	92
6.8	Automated workflow for processing, classifying, and reacting to schema changes	93
6.9	Example of a breaking change alert email.	95



List Of Tables

1.1	Product backlog for the Data Pipeline Platform	14
2.1	Comprehensive Comparison Between ETL and ELT Methodologies	22
4.1	Sprint 1 Backlog Items	46
4.2	Ingestion Tool Comparison by Feature	51
4.3	Ingestion Tool Initial Performance Benchmark (Time in Seconds)	51
4.4	Parallelized dlt vs. Apache NiFi Performance (Time in Seconds)	51
4.5	Sprint 2 Backlog Items	53
4.6	Transformation Tool Comparison by Feature	57
4.7	Transformation Performance Comparison (Execution Time in Seconds)	57
5.1	Sprint 3 Backlog Items	61
5.2	Sprint 4 Backlog Items	71
6.1	Sprint 5 Backlog Items	78
6.2	Sprint 6 Backlog Items	85



List of Abbreviations

API Application Programming Interface

ELT Extract Load Transform

ETL Extract Transform Load

LLM Large Language Model



General Introduction

Today, all businesses rely on data. Companies that can use their data well have a big advantage. But many companies still use old data systems that are slow, hard to manage, and cost a lot of money. This makes it hard for them to make quick decisions and keep up with changes.

This project, "A Modular Platform for Near-Real-Time Data Pipelin," solves this problem. Our main goal was to build a modern, flexible, and easy-to-use data platform that allows businesses to get the most out of their data. The platform has several key features:

- **User-Friendly Setup:** The system is designed so that even people who aren't technical experts can easily set up data pipelines.
- **Near-Real-Time Processing:** It processes data in minutes, ensuring that businesses are always working with the most current information.
- **Comprehensive Monitoring:** The system constantly watches for issues and sends alerts so problems can be fixed quickly, ensuring smooth operation.
- **Robust Security:** We built in strong security measures to protect sensitive data and ensure that only authorized users can access it.
- **Self-Service Analytics:** The platform empowers business users to create their own reports and dashboards, reducing their reliance on IT and freeing up technical teams to focus on more complex tasks.

By combining cutting-edge data technology with a focus on usability and security, this project provides a powerful solution to the challenges of traditional data systems.

The rest of this report is organized as follows:

- Chapter 1 provides a detailed overview of the project, including the context at Elyadata, the problem we addressed, and our work methodology.
- Chapter 2 is dedicated to the State of the Art and the different technologies used for data processing.
- Chapter 3 presents the conceptual design of the system, including its functional requirements and architecture
- Chapters 4, 5, and 6 detail the implementation phase, covering the development of the data ingestion, transformation, visualization, and security components.
- The General Conclusion summarizes our achievements, reflects on the project's outcomes, and discusses potential future work.

Preliminary Analysis

Contents

1.1	Introduction	4
1.2	General Presentation of the Organization	4
1.2.1	About Elyadata	4
1.2.2	Organizational Structure	5
1.2.3	Products and Services	5
1.3	Problem Statement	6
1.3.1	Current Data Processing Challenges	6
1.3.2	Limitations of Existing Solutions	6
1.3.3	Proposed Solution	7
1.4	Gantt Chart	8
1.5	Work Methodology	10
1.5.1	Scrum Framework Overview	10
1.5.2	Scrum Artifacts	11
1.5.3	Scrum Events	11
1.5.4	Scrum Roles	12
1.6	Version Control Strategy	12
1.7	Product Backlog	13
1.8	Conclusion	15

1.1 Introduction

This chapter situates the project within its broader context. It begins with a brief overview of the current state of the art in data engineering, which establishes the technical background for this work. The chapter then introduces the host organization, Elyadata, and examines its existing data processing methods to identify key limitations. Finally, it presents our proposed solution and outlines the agile work methodology adopted to guide the project's development and ensure its successful delivery.

1.2 General Presentation of the Organization

1.2.1 About Elyadata

Elyadata is a leading technology company founded in Tunisia in 2012, with offices in both Tunis and Paris. The company specializes in enabling businesses to leverage digital tools, data science, and artificial intelligence (AI) to enhance their operational efficiency. Since its inception, Elyadata has become a trusted partner for numerous public and private organizations, helping them achieve more informed decision-making through data-driven insights.

The company's identity is represented by its official logo, shown in Figure 1.1.



Figure 1.1: Elyadata's Logo

The core mission of Elyadata is to simplify complex systems by combining deep scientific expertise with a clear understanding of business needs. The company is recognized for its commitment to innovation, quality, and the delivery of high-impact solutions.

1.2.2 Organizational Structure

Elyadata is organized into several collaborative teams, each contributing specialized expertise:

- **Development Team:** Responsible for the end-to-end design, implementation, and maintenance of software solutions, from requirement analysis to final product deployment.
- **Data Science and AI Team:** Develops intelligent systems using machine learning, natural language processing, and computer vision to transform raw data into actionable insights.
- **Sales and Marketing Team:** Manages client relationships, identifies new business opportunities, and communicates the value of Elyadata's services to the market.
- **Support and Services Team:** Provides ongoing technical support and system maintenance to ensure client satisfaction and operational continuity.

1.2.3 Products and Services

Elyadata offers a comprehensive suite of services designed to advance digital transformation:

- **Custom Software Development:** Developing bespoke software solutions tailored to specific client requirements, from large-scale enterprise platforms to specialized tools.
- **Data Science and AI Solutions:** Creating applied AI systems that solve tangible business problems using advanced data-driven technologies.
- **IT Consulting:** Providing expert advisory services to help organizations optimize their IT infrastructure and align technology with strategic business goals.
- **Cloud Solutions:** Architecting and implementing cloud-based systems to enhance scalability, flexibility, and operational efficiency.
- **Data Annotation Services:** Labeling diverse datasets (speech, text, images, video) to train high-accuracy machine learning models.

- **Digital Transformation Strategy:** Guiding clients through automation, system integration, and data analysis to facilitate their adaptation to the digital landscape.

Furthermore, Elyadata actively participates in research and innovation through European projects and scientific publications, ensuring it remains at the cutting edge of technology and AI.

1.3 Problem Statement

1.3.1 Current Data Processing Challenges

Many organizations still rely on traditional data processing methods, such as manually coded, batch-oriented pipelines. While once suitable for an era of smaller data volumes and slower business cycles, these approaches are now ill-equipped to meet modern demands. They introduce significant challenges, including data latency and high maintenance costs, which limit organizational agility and impair decision-making in today's fast-paced environment.

1.3.2 Limitations of Existing Solutions

Traditional pipeline architectures present several critical limitations that impact business operations:

- **Data Latency:** Business intelligence dashboards and analytical reports frequently display information that is hours or even days old. This delay means decisions are often based on outdated information that no longer reflects current market conditions.
- **Delayed Decision-Making:** Without access to fresh data, teams cannot react swiftly to emerging trends, market shifts, or operational issues. This latency results in missed opportunities and slower responses to critical business events.
- **High Maintenance Overhead:** Data engineers spend a disproportionate amount of time on pipeline maintenance and troubleshooting, diverting focus from high-value activities like data modeling and analysis.

- **Poor Scalability:** Adding new data sources or modifying existing pipelines is a time-consuming and error-prone process. This rigidity makes it difficult for organizations to adapt to changing business requirements or scale their data operations effectively.
- **Resource Inefficiency:** Batch processing leads to inefficient resource allocation, with systems sitting idle between scheduled runs and then experiencing peak loads that can strain infrastructure.

1.3.3 Proposed Solution

To address these challenges, we propose the development of a modular, near-real-time data processing platform. Our solution is designed with the following core principles:

- **User-Friendly Configuration:** Enabling business users to configure and modify data workflows through intuitive interfaces, eliminating the need for extensive technical knowledge or manual coding.
- **Near-Real-Time Processing:** Ensuring data is ingested, processed, and made available for analysis within minutes, allowing organizations to act on current information.
- **Comprehensive Monitoring:** Providing real-time visibility into workflow health and performance, with automated alerts to facilitate rapid issue identification and resolution.
- **Robust Security:** Implementing strong security measures, including role-based access control (RBAC), to ensure only authorized personnel can access or modify sensitive data and systems.
- **Self-Service Analytics:** Empowering business users to create their own reports and dashboards without IT intervention, thereby increasing productivity and reducing the burden on technical teams.

This platform directly counters the limitations of traditional approaches by providing a flexible, efficient, and user-friendly system that enables organizations to fully leverage their data assets.

Of course. Here is a more concise version that briefly explains each task without unnecessary detail.

1.4 Gantt Chart

The project timeline was managed using a Gantt chart, providing a clear overview of task scheduling and progress across six months. The development was divided into two main phases covering initial setup and advanced integrations.

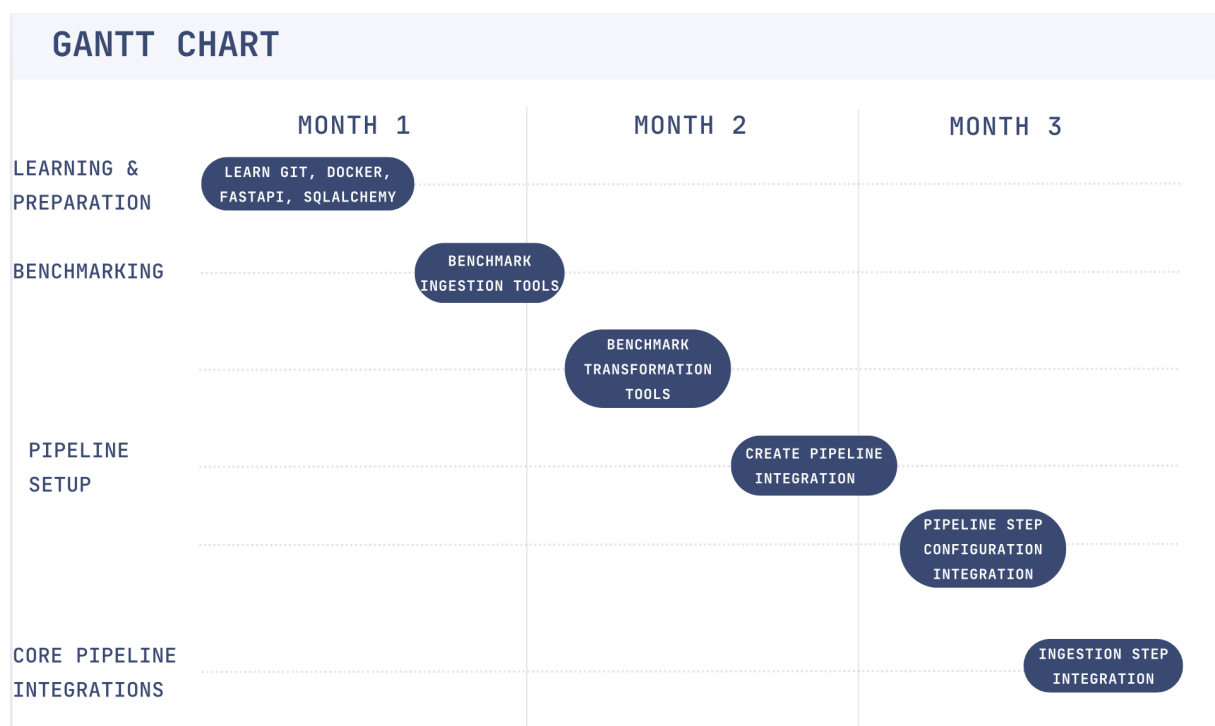


Figure 1.2: Project Timeline: Phase 1 (Months 1-3)

As shown in Figure 1.2, the first three months focused on foundation building and core development:

Month 1: Learning & Preparation

Acquired essential skills in Git, Docker, FastAPI, and SQLAlchemy to establish the project foundation.

Month 2: Benchmarking

Evaluated ingestion and transformation tools to select optimal solutions for data processing requirements.

Month 3: Pipeline Setup

Developed core pipeline architecture with configuration system and initial ingestion step integration.

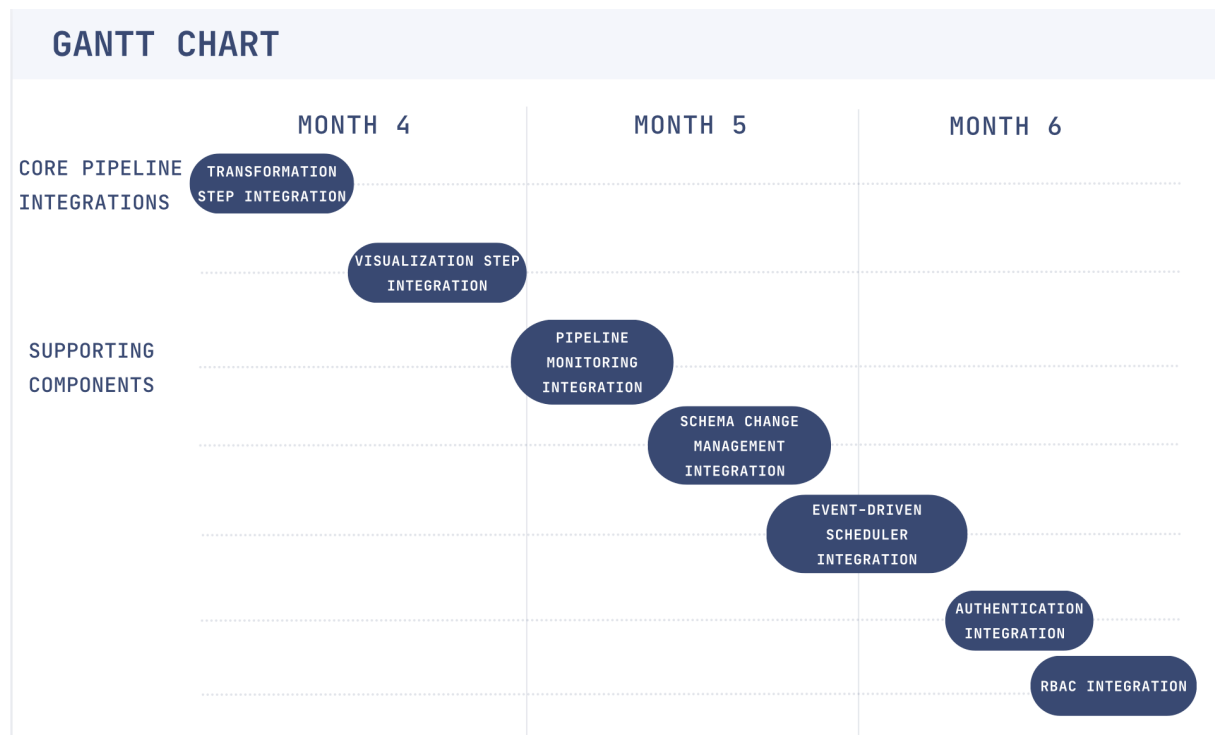


Figure 1.3: Project Timeline: Phase 2 (Months 4-6)

As shown in Figure 1.3, the final three months concentrated on core integrations and supporting components:

Month 4: Core Pipeline Integrations

Implemented transformation step for data processing and visualization step for report generation.

Month 5: Supporting Components

Added pipeline monitoring integration and schema change management tools for operational stability.

Month 6: Advanced Features

Integrated event-driven scheduler, authentication system, and role-based access control (RBAC) for security.

1.5 Work Methodology

1.5.1 Scrum Framework Overview

Scrum is an agile project management framework that enables teams to deliver value iteratively while adapting to changing requirements [4]. Rather than extensive upfront planning, Scrum divides work into manageable iterations called Sprints. This approach promotes collaboration, regular stakeholder feedback, and continuous improvement.

The cyclical process flow of the Scrum framework is illustrated in Figure 1.4, highlighting its key events such as Sprint Planning, Daily Scrums, Sprint Review, and Sprint Retrospective.

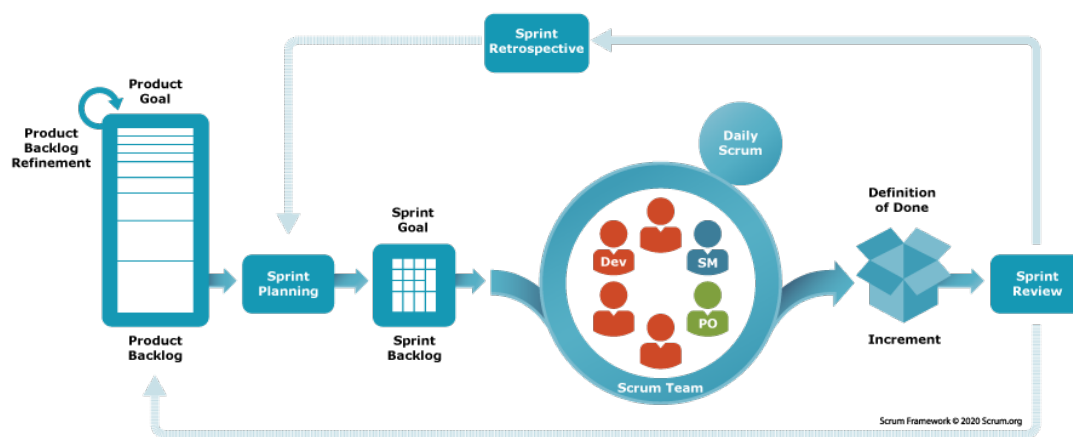


Figure 1.4: Scrum Framework Process Flow

Scrum was selected for this project due to its key advantages:

- **Flexibility and Adaptability:** Allows the team to respond quickly to evolving technical and business needs.
- **User-Centered Focus:** Ensures development priorities are aligned with user requirements and business value.
- **Manageable Work Organization:** Breaks down complex development into smaller, achievable objectives.

- **Efficient Resource Management:** Optimizes time and resource allocation throughout the development lifecycle.
- **Continuous Improvement:** Incorporates regular feedback loops for process and product refinement.

1.5.2 Scrum Artifacts

Scrum artifacts are "information radiators" that provide transparency into the project's status and value. They are essential for managing the development process effectively [4]:

- **Product Backlog:** A dynamic, ordered list of all features, enhancements, and technical requirements needed for the product. It is continuously refined based on stakeholder feedback and business priorities.
- **Sprint Backlog:** A subset of Product Backlog items selected for completion during a specific Sprint, plus a plan for delivering them. It is owned and managed by the Development Team.
- **Product Increment:** The sum of all Product Backlog items completed during a Sprint, integrated with work from all previous Sprints. Each increment must be potentially releasable and meet the team's Definition of Done.

1.5.3 Scrum Events

Scrum defines several time-boxed events to create regularity and enable transparency, inspection, and adaptation [4]:

- **The Sprint:** A fixed-length event of one month or less during which a "Done," usable, and potentially releasable product Increment is created. This project utilizes two-week Sprints.
- **Sprint Planning:** A collaborative meeting at the start of each Sprint to define the Sprint Goal and select the work to be performed from the Product Backlog.

- **Daily Scrum:** A 15-minute daily meeting for the Development Team to synchronize activities and create a plan for the next 24 hours.
- **Sprint Review:** An event held at the end of the Sprint to inspect the Increment and adapt the Product Backlog if needed. The team demonstrates completed work to stakeholders.
- **Sprint Retrospective:** An opportunity for the Scrum Team to inspect itself and create a plan for improvements to be enacted during the next Sprint.

1.5.4 Scrum Roles

The Scrum framework defines three roles, each with distinct responsibilities [4]:

- **Product Owner:** Responsible for maximizing the value of the product resulting from the work of the Development Team. The Product Owner manages the Product Backlog and represents stakeholder interests. In our project, this role was assumed by **Nihed Moshabhi**.
- **Scrum Master:** A servant-leader for the Scrum Team, responsible for promoting and supporting Scrum. The Scrum Master helps remove impediments to the team's progress. In our case, the role of Scrum Master was taken by **Amal Benyakhlef**.
- **Development Team:** A self-organizing, cross-functional group of professionals who do the work of delivering a potentially releasable Increment of "Done" product at the end of each Sprint. For this project, I, **Wisse Hamoudi**, played the role of the Development Team.

1.6 Version Control Strategy

To complement the Scrum framework, we adopted the **GitFlow** branching model for version control. This technical workflow provides a robust structure that aligns with our agile process. Each user story from the Sprint Backlog is developed in an isolated **feature** branch, ensuring

that ongoing work does not destabilize the main codebase. At the end of each Sprint, completed features are merged into the **develop** branch, which represents the cutting-edge version of our Product Increment.

As illustrated in Figure 1.5, GitFlow also provides dedicated **release** branches for preparing public versions and **hotfix** branches for addressing urgent production issues. This methodology ensures a clean project history and supports parallel development, which is critical for maintaining development velocity while ensuring production stability.

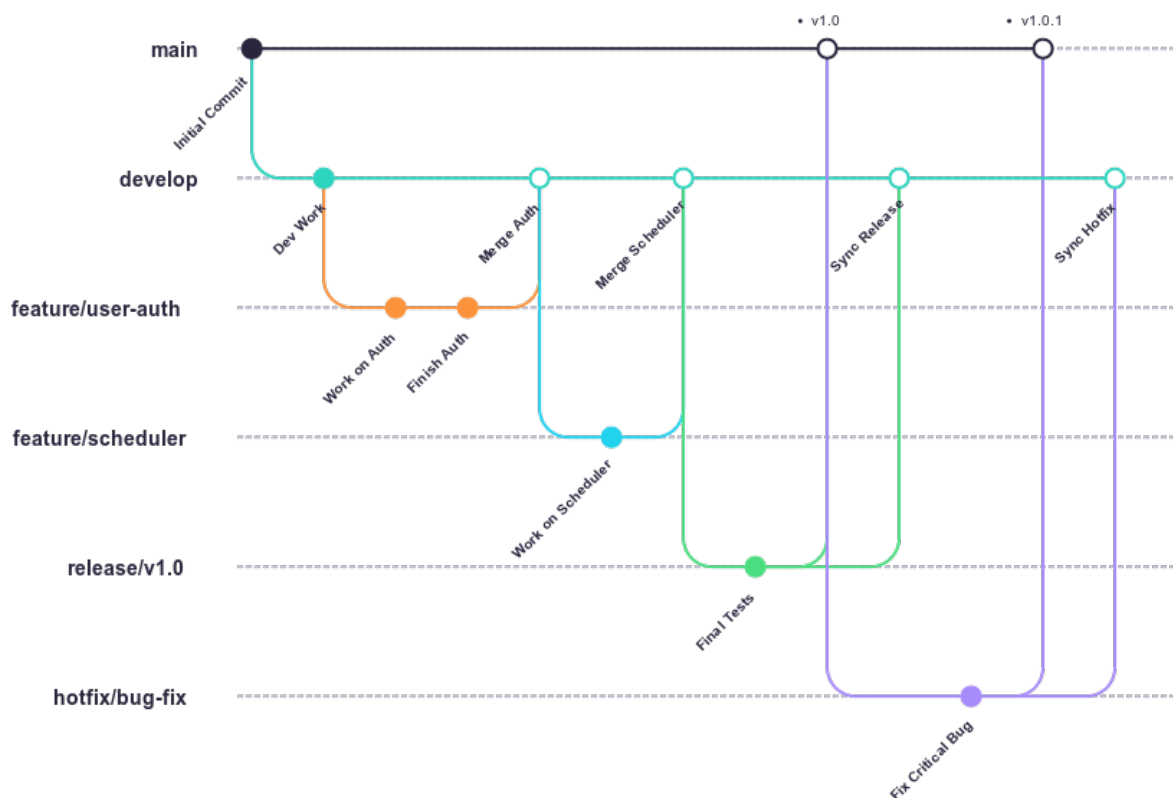


Figure 1.5: The GitFlow Branching Model

1.7 Product Backlog

The product backlog outlines the development roadmap for the platform, with features prioritized based on their contribution to core functionality. Table 1.1 presents the complete backlog, including features, user stories, and priority levels.

Table 1.1: Product backlog for the Data Pipeline Platform

ID	Feature	User Story	Priority
1	Authentication System	As a user , I want to log in securely to the platform, so that I can access its features and protect my data from unauthorized access.	High
2	Step Configuration Framework	As an administrator , I want to configure pipeline steps with minimal effort, so that I can accelerate the deployment of new workflows.	Medium
3	Ingestion Step	As an administrator , I want to add a pre-built ingestion component to a pipeline, so that I can easily connect to and extract data from a source system.	Medium
4	Transformation Step	As an administrator , I want to add a pre-built transformation component to a pipeline, so that I can apply data cleaning or enrichment rules efficiently.	Medium
5	Interactive Visualization	As a user , I want to explore pipeline results through interactive charts, so that I can quickly identify trends and insights.	Medium
<i>Continued on next page</i>			

Table 1.1 continued from previous page

ID	Feature	User Story	Priority
6	Event-Driven Scheduler	As an administrator , I want pipelines to run automatically when a specific event occurs, so that data is processed in near-real-time without manual intervention.	High
7	Schema Evolution Management	As an administrator , I want the system to alert me to structural changes in incoming data, so that I can prevent pipeline failures when a source is updated.	Low
8	Pipeline Monitoring	As an administrator , I want to monitor the health, status, and performance of all running pipelines, so that I can proactively identify and resolve issues.	Low
9	Role-Based Access Control	As an administrator , I want to assign roles and permissions to users, so that I can enforce security policies and control access to data.	Low

1.8 Conclusion

In summary, traditional data processing systems are failing to meet modern business demands, as they are burdened by high latency and significant maintenance overhead. This project directly addresses these shortcomings by proposing a near-real-time, user-friendly data pipeline platform. By adopting the Scrum framework, we ensure an adaptive, iterative development process focused

on delivering value and incorporating user feedback. This synthesis of modern data engineering principles and agile methodology positions the project to deliver a robust and scalable solution, empowering organizations to overcome the critical data challenges of today's business environment.

State of the Art

Contents

2.1	Introduction	18
2.2	Introduction to Modern Data Engineering	18
2.3	ETL vs. ELT: Core Data Processing Paradigms	19
2.3.1	Overview of Data Integration Strategies	19
2.3.2	Extract, Transform, Load (ETL) Methodology	19
2.3.3	Extract, Load, Transform (ELT) Methodology	21
2.3.4	Comparative Analysis and Decision Framework	22
2.3.5	Hybrid Approaches and Modern Implementations	22
2.4	Data Pipeline Scheduling and Orchestration	23
2.4.1	The Evolution from Batch to Event-Driven Processing	23
2.4.2	Core Concepts of Event-Driven Architecture	23
2.4.3	Implementation: Patterns and Technology Stack	25
2.4.4	Challenges and Future Directions	27
2.5	Conclusion	28

2.1 Introduction

The efficient processing of massive data volumes presents a significant challenge for modern organizations. This chapter examines the current landscape of data engineering to analyze the capabilities and limitations of existing technologies. We analyze three key areas: data integration approaches (ETL vs. ELT), the evolution of pipeline scheduling from batch to event-driven processing, and the role of modern orchestration tools. This analysis identifies critical gaps in current solutions and establishes the technical rationale for the proposed system in this work.

2.2 Introduction to Modern Data Engineering

Data engineering has emerged as a critical discipline in the era of big data and digital transformation [1]. As organizations increasingly rely on data-driven decision-making, the need for robust, scalable, and efficient data processing systems has become paramount. Data engineering encompasses the design, construction, and maintenance of systems that collect, store, process, and serve data to support various analytical and operational use cases.

The field of data engineering involves multiple interconnected components that form comprehensive data ecosystems. Data engineers are responsible for building data pipelines—automated workflows that facilitate the movement of data from various sources to designated destinations such as data warehouses, data lakes, or real-time analytics platforms.

Modern data engineering has evolved significantly from traditional batch-processing approaches to incorporate real-time streaming, cloud-native architectures, and event-driven paradigms. This evolution is driven by changing business requirements, technological advances, and the exponential growth in data volume, velocity, and variety that organizations must now handle.

2.3 ETL vs. ELT: Core Data Processing Paradigms

2.3.1 Overview of Data Integration Strategies

In the landscape of modern data engineering, two dominant strategies have emerged for building efficient data pipelines: **Extract, Transform, Load (ETL)** and **Extract, Load, Transform (ELT)** [2]. These methodologies represent distinct philosophical approaches to data processing, with significant implications for system architecture, performance, and operational efficiency.

The choice between ETL and ELT has become increasingly important as organizations scale their data operations and adopt cloud-based infrastructure. A clear understanding of the nuances, benefits, and trade-offs of each approach is essential for designing systems that meet specific organizational requirements.

Figure 2.1 illustrates the fundamental architectural differences between ETL and ELT workflows, highlighting the key distinction in when and where data transformation occurs within the pipeline.

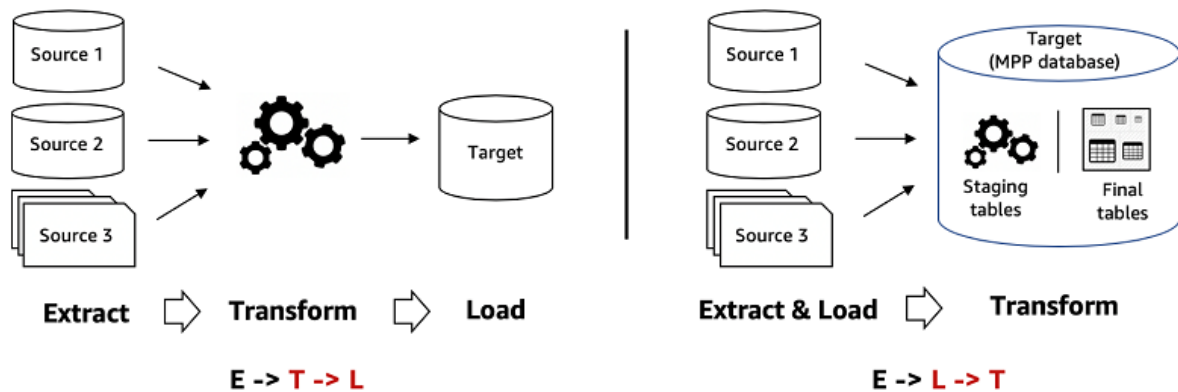


Figure 2.1: Comparative Workflow Architecture of ETL and ELT Data Processing Pipelines

2.3.2 Extract, Transform, Load (ETL) Methodology

2.3.2.1 ETL Process Definition and Architecture

The ETL (Extract, Transform, Load) methodology is the traditional approach to data integration and has been widely adopted in enterprise data warehousing for decades [2]. This approach

uses a sequential model where data transformation occurs on dedicated processing servers or specialized ETL tools before the data reaches its target destination.

The ETL process consists of three distinct phases:

1. **Extract Phase:** Data is retrieved from multiple heterogeneous source systems, such as relational databases, flat files, APIs, and legacy applications.
2. **Transform Phase:** The raw extracted data undergoes comprehensive processing to meet business and analytical requirements. This phase includes data cleansing, standardization, validation, enrichment, and aggregation.
3. **Load Phase:** The fully processed and transformed data is loaded into the target data warehouse or analytical database.

2.3.2.2 Advantages of the ETL Approach

The ETL methodology offers several key advantages:

- **Data Quality and Governance:** ETL provides extensive control over data quality and validation before information enters the data warehouse, which is crucial for maintaining data integrity [1].
- **Regulatory Compliance:** Industries with strict regulatory requirements benefit from ETL's capacity to implement rigorous data validation, masking, and audit trails prior to loading.
- **Legacy System Integration:** ETL tools are often well-suited for organizations with established on-premise infrastructure and legacy data sources.

2.3.3 Extract, Load, Transform (ELT) Methodology

2.3.3.1 ELT Process Definition and Modern Architecture

The ELT (Extract, Load, Transform) methodology represents a paradigm shift enabled by advances in cloud computing and modern data platforms. This approach leverages the computational power of the target data warehouse to perform transformations directly on the stored data.

The ELT process follows a modified sequence:

1. **Extract Phase:** Data is collected from various source systems.
2. **Load Phase:** Raw, unprocessed data is loaded directly into a data warehouse or data lake, often in its native format.
3. **Transform Phase:** Data transformation occurs within the data warehouse using SQL-based operations or integrated cloud-native tools.

2.3.3.2 Advantages of the ELT Approach

The ELT methodology offers compelling benefits for modern data stacks:

- **Scalability and Performance:** ELT leverages the elastic scalability of cloud-native data warehouses like Snowflake, Amazon Redshift, and Google BigQuery to process massive datasets efficiently.
- **Cost Optimization:** By eliminating the need for separate ETL processing servers, organizations can reduce infrastructure complexity and operational costs.
- **Near-Real-Time Processing:** ELT architectures are better suited for streaming data and real-time analytics, as they reduce the latency between data arrival and availability.
- **Flexibility and Agility:** This approach enables more flexible data exploration and iterative development, as analysts can work with raw data directly before transformations are defined.

2.3.4 Comparative Analysis and Decision Framework

Table 2.1 provides a comprehensive comparison of the ETL and ELT approaches across multiple critical dimensions.

Table 2.1: Comprehensive Comparison Between ETL and ELT Methodologies

Criteria	ETL Approach	ELT Approach
Transformation Timing	Before data loading	After data loading
Processing Location	External ETL servers/tools	Within the data warehouse
Infrastructure Type	On-premise and hybrid	Cloud-native and optimized
Performance with Large Datasets	Limited by processing server capacity	Leverages distributed cloud processing
Scalability	Constrained by dedicated hardware	Highly scalable with cloud resources
Data Quality Control	Extensive pre-load validation	Post-load validation and monitoring
Time to Market	Longer development cycles	Faster implementation and iteration

2.3.5 Hybrid Approaches and Modern Implementations

In practice, contemporary data engineering often employs hybrid strategies that combine elements of both ETL and ELT. For instance, an organization might use ETL for sensitive data requiring extensive pre-load validation while using ELT for exploratory analytics on less structured data. This flexibility allows architectural designs to be optimized for diverse use cases and business requirements.

2.4 Data Pipeline Scheduling and Orchestration

2.4.1 The Evolution from Batch to Event-Driven Processing

The orchestration and scheduling of data workflows are critical components of modern data engineering. As pipelines have evolved from simple, monolithic batch jobs to complex, interdependent workflows, the sophistication of scheduling mechanisms has increased accordingly.

Traditional data processing relied heavily on time-based schedulers, such as cron jobs. While predictable, these approaches often led to inefficient resource utilization and delayed data availability. The evolution toward more dynamic, responsive scheduling has been driven by business demands for real-time insights and operational efficiency. Consequently, modern data engineering has embraced event-driven architectures that respond dynamically to data and system events, representing a fundamental shift from rigid schedules to flexible, condition-based execution.

2.4.2 Core Concepts of Event-Driven Architecture

Event-driven scheduling is a paradigm where pipeline operations are triggered in response to specific events rather than at predetermined intervals. This approach enables more responsive, efficient, and resilient data workflows that adapt to real-time conditions. The fundamental principle is that processing should occur precisely when it is needed, which reduces both computational overhead and data latency.

Event-driven architectures are built upon several core concepts:

- **Event Producers:** Systems that generate events (e.g., database transactions, file uploads, API calls).
- **Event Brokers (Message Queues):** Middleware components that receive, queue, and route events to consumers.

- **Event Consumers:** Components or services that subscribe to specific events and execute actions when triggered.

2.4.2.1 Event Types and Triggering Mechanisms

Event-driven schedulers can monitor and respond to various categories of events:

- **Data Availability Events:** Indicate that new data is ready for processing (e.g., new file uploads, database record insertions, API updates).
- **System State Events:** Reflect changes in system status (e.g., server status changes, resource utilization crossing a threshold, completion of an upstream task).
- **Business Logic Events:** Generated based on predefined business rules (e.g., a customer transaction exceeding a certain value, inventory levels falling below a set point).
- **Monitoring and Quality Events:** Indicate data quality issues or system performance degradation (e.g., schema validation failures, high error rates).

2.4.2.2 Event Processing and Decision Logic

Modern event-driven schedulers implement sophisticated logic that extends beyond simple triggers [3]. These systems must evaluate multiple factors:

- **Event Filtering and Classification:** Schedulers apply filtering logic to determine which events are relevant for specific workflows, ignoring noise.
- **Event Correlation and Aggregation:** Many workflows require multiple conditions to be met. Schedulers can correlate events from different sources to satisfy complex triggering criteria.
- **Dependency Resolution:** Schedulers must understand and manage dependencies between tasks, ensuring all prerequisites are satisfied before initiating downstream processing.

2.4.3 Implementation: Patterns and Technology Stack

2.4.3.1 Common Implementation Patterns

Event-driven scheduling is implemented through several established design patterns:

- **File Watcher Pattern:** Monitoring file systems or cloud storage (e.g., AWS S3) for new files and triggering extraction processes upon their arrival.
- **Change Data Capture (CDC) Pattern:** Capturing row-level database changes in real-time to trigger transformations for modified records.
- **Webhook Integration Pattern:** Configuring external systems to send notifications (webhooks) to an endpoint when specific events occur.
- **Stream Processing Pattern:** Using platforms like Kafka or Kinesis to trigger processes based on data volume, content, or time-based windows.
- **Quality Gate Pattern:** Triggering downstream tasks only after data quality validation on a source dataset has completed successfully.

2.4.3.2 The Technology Stack: Layers and Tools

Event-driven systems typically consist of two primary, complementary layers. The **Event Infrastructure Layer** is the messaging backbone for event generation and distribution. The **Orchestration Layer** sits above it, providing intelligent workflow management and executing business logic in response to events.

Orchestration Platforms.

These platforms are the foundation for managing event-driven workflows.

- **Apache Airflow** is a widely adopted open-source platform that enables event-driven workflows through its sensor framework and dataset-driven scheduling [3].

- **Prefect** offers a modern, Python-native approach with automatic dependency resolution and robust error handling capabilities.
- **Dagster** implements an asset-centric model where workflows are organized around data assets, which aligns naturally with event-driven principles.

Cloud-Native Orchestration Solutions.

Major cloud providers offer managed services that integrate deeply with their ecosystems.

- **Azure Data Factory** provides enterprise-scale data integration with multiple trigger types (e.g., storage events, custom events).
- **AWS Step Functions** enables serverless workflow orchestration using state machines, with native integration to services like Lambda and S3.
- **Google Cloud Composer** provides managed Apache Airflow with deep integration into the Google Cloud Platform.

Event Infrastructure and Messaging Systems.

This infrastructure is required to handle event streams reliably and at scale.

- **Apache Kafka** is a cornerstone of many architectures, providing high-throughput, fault-tolerant event streaming.
- **Amazon Kinesis** and **Google Cloud Pub/Sub** offer managed, serverless streaming services optimized for their cloud environments.

Serverless Computing Platforms.

These platforms provide scalable, cost-effective execution environments for event-driven functions.

- **AWS Lambda, Azure Functions, and Google Cloud Functions** allow developers to execute code in response to events without managing servers, automatically scaling with event volume.

2.4.4 Challenges and Future Directions

2.4.4.1 Implementation Challenges

Despite their advantages, implementing robust event-driven systems presents several technical challenges:

- **Event Ordering and Consistency:** Ensuring data consistency across distributed systems requires careful event sequencing, idempotency controls, and conflict resolution strategies.
- **Error Handling and Recovery:** Sophisticated error handling, such as dead-letter queues and configurable retry policies, is necessary to manage failures gracefully.
- **Scalability and Performance:** Systems must be architected to scale both event processing and downstream resources as data volumes increase.
- **Testing and Validation:** The non-deterministic nature of event-driven systems requires specialized testing approaches, such as event replay and chaos engineering.
- **Monitoring and Observability:** The dynamic and distributed nature of these systems makes troubleshooting complex, demanding comprehensive logging and distributed tracing.

2.4.4.2 Future Trends in Data Engineering

The field continues to evolve rapidly with several emerging trends:

- **Predictive Scheduling:** Machine learning models are being integrated to anticipate events and preposition resources, optimizing allocation and reducing latency.
- **Serverless and Edge Computing:** The expansion of serverless and edge computing extends event-driven processing to distributed environments closer to data sources.

- **Autonomous Data Operations (DataOps):** AI/ML integration is driving a move toward autonomous systems that can self-heal, self-optimize, and adapt to changing conditions with minimal human intervention.
- **Complex Event Processing (CEP):** CEP capabilities are becoming more accessible, enabling sophisticated, real-time pattern detection across multiple event streams.

2.5 Conclusion

This chapter has explored key paradigms and advancements in modern data engineering, charting the industry's shift from rigid batch processing to dynamic, event-driven architectures. While ETL and ELT both offer distinct strengths, the demands of real-time applications and massive data volumes necessitate more adaptable and responsive solutions. The analysis of current orchestration tools and their underlying challenges reveals a clear need for innovation, particularly in simplifying the construction of scalable and resilient data workflows. This identified gap provides the motivation for the system proposed in the following chapters.

Project Scope And Conceptual Design

Contents

3.1	Introduction	30
3.2	System Requirements	30
3.2.1	Functional Requirements	30
3.2.2	Non-Functional Requirements	33
3.3	Global Use Case Diagram	34
3.4	Functional Modeling	35
3.4.1	Solution Overview	35
3.4.2	Class Diagram	36
3.4.3	C4 Models	37
3.4.4	Sequence Diagrams	40
3.5	Technology Stack and Work Environments	41
3.5.1	Development and Collaboration Tools	41
3.5.2	Backend Technology	41
3.5.3	Frontend Technology	43
3.5.4	Database and Storage	43
3.5.5	Containerization and Deployment	43
3.6	Conclusion	44

3.1 Introduction

This chapter provides a thorough conceptual analysis of the data pipeline platform, establishing a formal blueprint before implementation begins. This analysis is a critical phase in software engineering, as it translates abstract goals into a concrete and verifiable design. The chapter begins by defining the core functional and non-functional requirements that the system must satisfy. Subsequently, it models the system's architecture and behavior from multiple perspectives using standard methodologies: Use Case diagrams for user interactions, the C4 model for architectural decomposition, and UML diagrams (Class and Sequence) for static structure and dynamic behavior. Finally, it outlines the software environments and frameworks selected for the development process. This chapter lays the essential groundwork for the detailed design and implementation phases that follow.

3.2 System Requirements

To build an effective data pipeline platform, it is necessary to define precisely what the system should do (functional requirements) and its operational qualities (non-functional requirements). This process is vital for establishing a clear scope, guiding development efforts, and forming the basis for future testing and validation. This section outlines the specific requirements that will guide the development process.

3.2.1 Functional Requirements

Functional requirements describe the specific behaviors, features, and functions the system must perform. They represent a contract that defines the tangible deliverables of the project, ensuring that the final product meets the stakeholders core needs.

3.2.1.1 Pipeline Management

To provide a complete operational toolkit, the platform must empower users with full control over the entire lifecycle of a data pipeline. The essential management capabilities include:

- Users must be able to create new data pipelines via an intuitive interface with minimal configuration.
- Users must be able to modify existing pipelines without interrupting other running processes.
- Users must be able to delete pipelines they no longer need.
- Users must be able to view a list of all their pipelines with current status information.

3.2.1.2 Data Ingestion

A core function of the platform is its ability to connect to diverse data ecosystems. Therefore, the system must be able to handle different types of data sources:

- Support for structured data.
- Support for semi-structured data.

3.2.1.3 Data Transformation

The platform provides powerful data processing capabilities that allow users to clean, modify, and improve their data within the workflow. These capabilities include:

- **Column transformations:** Change data types, clean text, and perform calculations.
- **Table joining:** Combine multiple datasets by matching common fields, supporting various join types (inner, outer, left, right).
- **Smart transformations:** AI-powered transformations controlled through natural language commands for data cleaning.

3.2.1.4 Visualization Integration

To deliver value, processed data must be easily consumable. The platform must enable users to visualize processed data in meaningful formats:

- Integration with data visualization tools (e.g., Apache Superset) to create and embed visualizations.
- Support for automatic dashboard updates as new data flows through pipelines.
- Implementation of role-based access control (RBAC) for dashboards to ensure data security.

3.2.1.5 Event-Driven Execution

To ensure timely data processing and efficient resource use, the platform must run pipelines automatically and efficiently based on real-world triggers:

- Trigger pipeline execution when new data becomes available in a source system.
- Implement automatic retry mechanisms with configurable policies for failed pipeline runs.

3.2.1.6 Schema Change Management

Data sources evolve over time, and the platform must handle changes in data structure gracefully to prevent pipeline failures. The system must provide:

- Automatic detection of structural changes in source data before each pipeline run.
- Proactive user alerts when schema changes are detected, allowing for manual intervention.

3.2.1.7 Monitoring and Observability

A reliable system must be transparent. The platform must provide complete visibility into its data processing operations:

- Real-time display of pipeline execution status (e.g., running, success, failed).
- Historical tracking of pipeline performance and run durations over time.
- A centralized dashboard showing overall system health and recent activity.

3.2.1.8 Role-Based Access Control

To maintain security and operational integrity, different users require different levels of access.

The system will implement a clear separation of roles:

- **Administrator (Admin):** Full system access, including user management, system-level configuration, and all pipeline operations.
- **User (Viewer):** Can view and execute assigned pipelines, access monitoring dashboards, and review results, but cannot create or modify pipeline configurations.

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, specifying how well it performs its functions. These requirements are crucial for ensuring a positive user experience, system reliability, and long-term maintainability.

3.2.2.1 Performance Requirements

The system must be responsive and efficient to support near-real-time business operations.

These benchmarks ensure a fluid user experience and timely data delivery:

- Data ingestion and basic transformations should complete within one minute for typical datasets (up to 100,000 records).
- Dashboard and visualization updates should appear within 30 seconds of data processing completion.

3.2.2.2 Scalability Requirements

The platform must be architected to be extensible, allowing it to grow with user needs and evolving technology without requiring a complete redesign:

- The system must support the addition of new data source types through a modular plugin architecture.
- The system must allow for the integration of new transformation functions without modifying core system code.
- The system must be able to connect with new visualization and analysis tools via a generic export mechanism.

3.2.2.3 Usability Requirements

To maximize adoption, the system must be accessible and intuitive for users with varying technical backgrounds:

- An intuitive web-based interface that is responsive on modern desktop and tablet devices.
- A non-expert user should be able to create and run a simple pipeline after a 30-minute introductory training session.

3.3 Global Use Case Diagram

A Use Case diagram models the system's functionality from a user's perspective. The diagram, shown in Figure 3.1, defines the primary actors and their allowed interactions with the system, effectively mapping user goals to system features.

- **User:** Can manage pipeline runs (execute, stop), monitor pipeline status, and view historical execution logs.

- **Admin:** Possesses all User capabilities, in addition to managing the pipelines themselves (create, edit, delete), user accounts, and system-wide access controls.

All actions require authentication, and access to specific functionality is determined by the user's assigned role, which is verified upon every interaction.

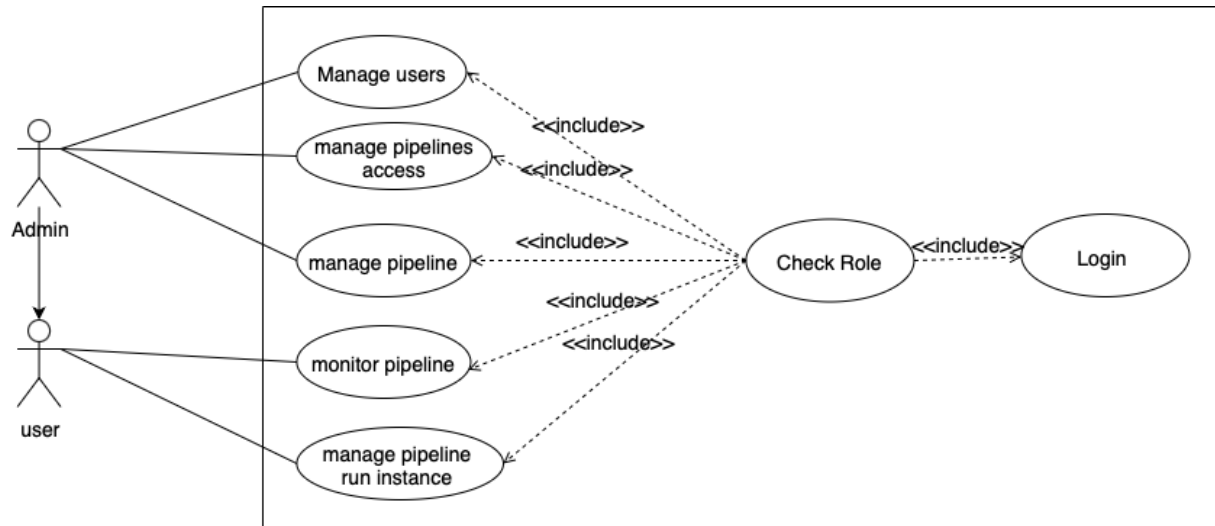


Figure 3.1: Global Use Case Diagram

3.4 Functional Modeling

This section translates the abstract requirements into concrete design models. Using a combination of standard modeling notations, we visualize the system's architecture, user interactions, and data structures to create a comprehensive conceptual blueprint.

3.4.1 Solution Overview

Our solution is a streamlined pipeline construction platform designed to democratize data engineering. It transforms complex, code-heavy tasks into an intuitive, configurable workflow. The core value proposition is the reduction of manual effort and technical barriers, enabling users to connect data sources, apply processing logic, and deliver analytics with unprecedented speed. By minimizing the setup overhead of traditional systems, the platform supports decision-making on live data streams with sub-second latency.

The high-level architecture of this solution is depicted in Figure 3.2.

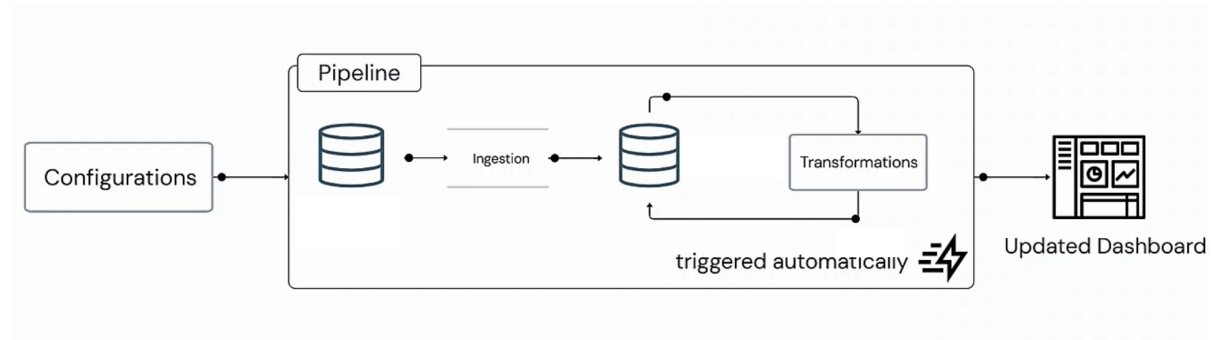


Figure 3.2: Solution Overview

3.4.2 Class Diagram

The class diagram provides a static blueprint of the system's structure, defining the data it manages and the relationships between different data objects. It is essential for database design and for understanding the core entities of the domain. This comprehensive diagram, shown in Figure 3.3, details the key components across user management, pipeline orchestration, and data processing.

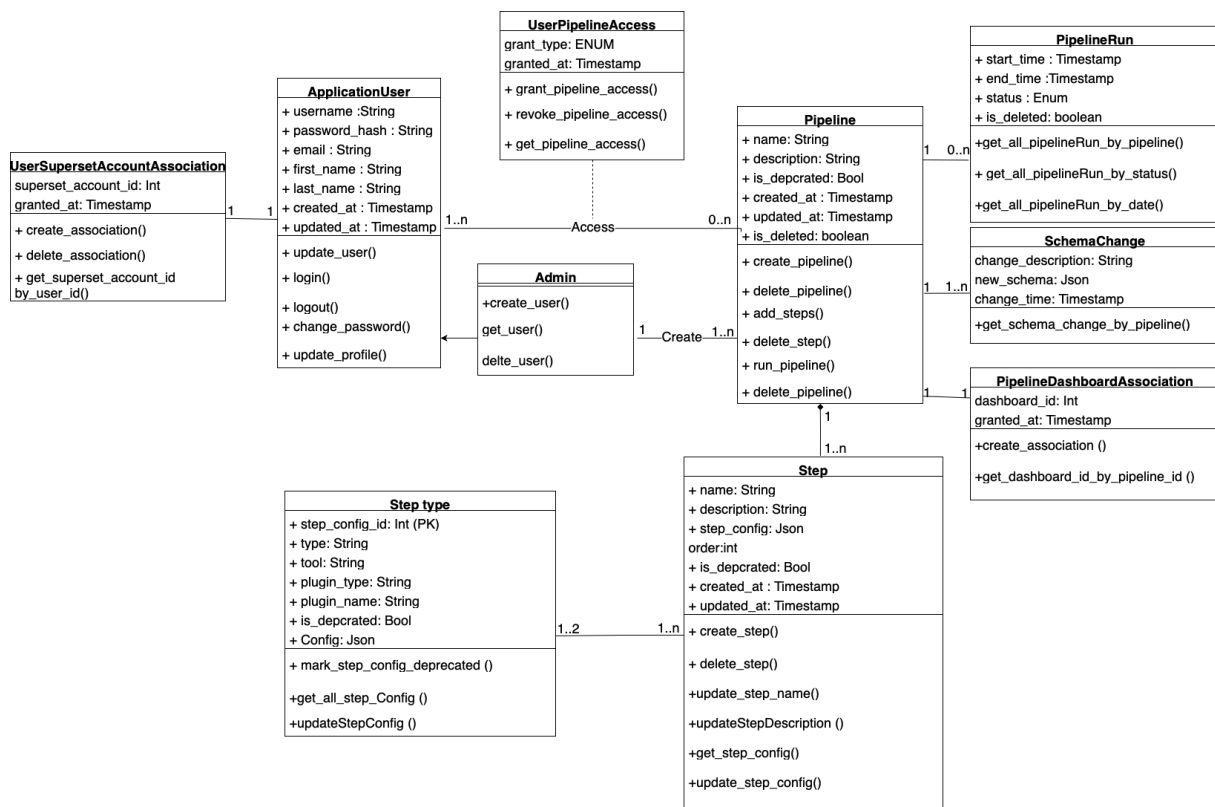


Figure 3.3: Class Diagram – System Architecture

- The system's users are represented by two distinct entities: Admin and User. The Admin class has elevated permissions and manages the User entities.
- The Pipeline entity holds the static definition of a workflow, while the PipelineRun entity records each specific execution. This one-to-many relationship creates a full historical log of every run for each pipeline.
- A Pipeline is composed of multiple Step entities, forming a one-to-many relationship that defines the sequence of operations in a workflow. The Step entity's behavior is defined by configurable JSON parameters and is associated with a StepType entity, promoting reusability.
- The UserPipelineAccess table acts as a many-to-many link between User and Pipeline, enabling granular permissions that define which users can interact with specific pipelines.
- For data governance and debugging, the SchemaChange table logs all detected structural changes in source data, providing a crucial audit trail when pipelines fail or behave unexpectedly. It is directly linked to the PipelineRun entity.
- Finally, the PipelineDashboardAssociation entity serves as a bridge, explicitly linking the output of a Pipeline to a corresponding dashboard in Superset. This formalizes the connection between the data processing and visualization layers.

3.4.3 C4 Models

To provide a clear, hierarchical view of the system's architecture, we use the C4 model. This model visualizes software at different levels of abstraction (Context, Containers, Components, Code), making it easier for diverse stakeholders to understand. The following sections present the first three levels.

3.4.3.1 System Context Diagram

The System Context diagram (Figure 3.4) provides the highest-level, "big picture" view of the project. It shows our entire Pipeline Manager system as a single box to clearly define its

boundaries. This simple diagram helps everyone understand the system's main connections to the people who use it and the external services it depends on.

In the diagram, we can see the two types of users: the User and the Admin. We can also see a very important connection to an external system: a GitHub repository. This is where the code for each data pipeline is stored.

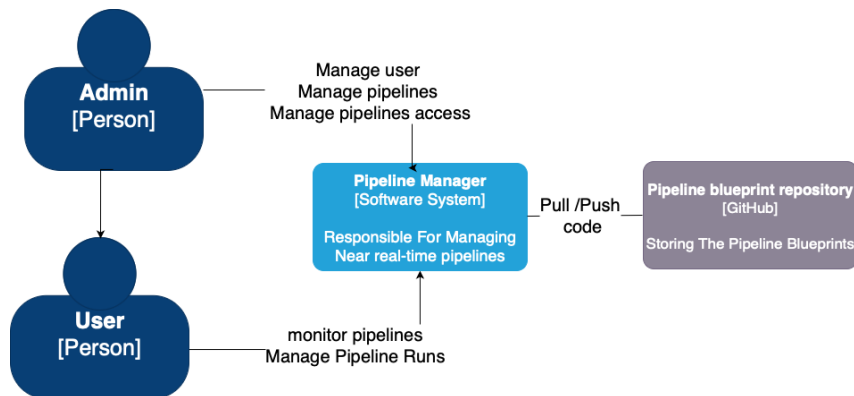


Figure 3.4: System context diagram

3.4.3.2 Container Diagram

Zooming into the Pipeline Manager, the Container diagram (Figure 3.5) reveals the high-level technological building blocks. Our architecture consists of three such containers, each with a distinct responsibility:

- A **Frontend** web application built with React.js, which runs in the user's browser and is responsible for all user interface rendering and interaction.
- A **Backend API** built with FastAPI (Python), which serves as the central engine for the platform. It handles business logic, orchestrates pipeline execution, and manages all data.
- A **PostgreSQL Database** for persistent storage, which holds all application data.

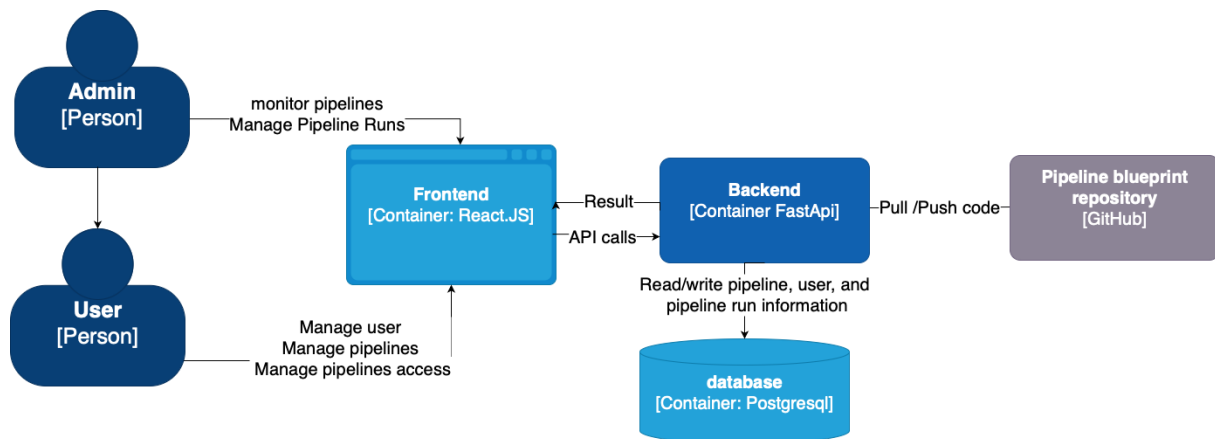


Figure 3.5: Container diagram

3.4.3.3 Component Diagram

Zooming further into the Backend container, the architecture is divided into several key components, each with a distinct responsibility. This modular design follows the principle of separation of concerns:

User access is governed by two distinct security modules. The **Authentication** component manages user login by verifying their identity. Once authenticated, the **Authorization** component enforces access rights, ensuring users can only perform actions and view data permitted by their assigned role.

The core business entities are handled by dedicated managers. The **User Management** component is responsible for the full lifecycle of user accounts, including their creation, updating, and deletion. The central orchestrator is the **Pipeline Management** component, which handles all operations related to the data pipelines themselves, including their creation, configuration, and execution.

To interface with the technical environment, two specialized components are used. The **Schema Change Manager** monitors for structural changes in source databases (e.g., new tables or columns) and reacts accordingly to prevent pipeline failures. Finally, the **Docker Engine Integration** component executes a pipeline's configuration by translating it into a running, isolated container, ensuring that every run is consistent and secure.

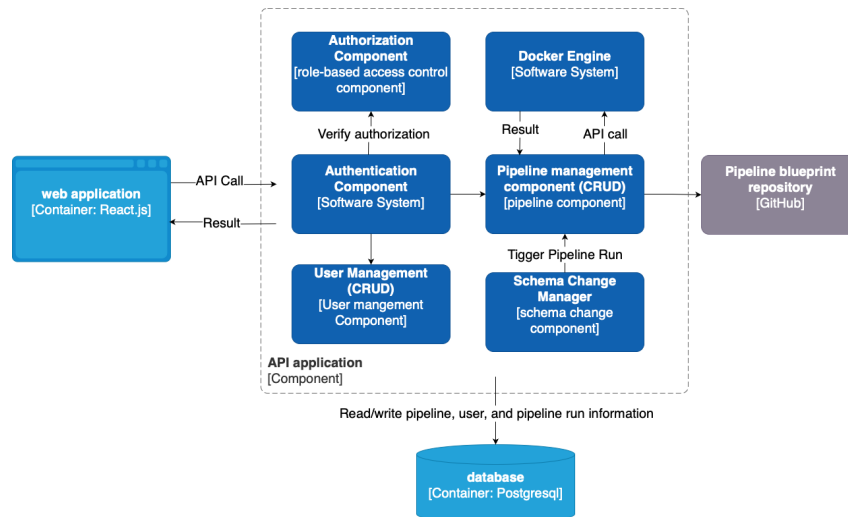


Figure 3.6: Component diagram

3.4.4 Sequence Diagrams

While class diagrams show a system's static structure, sequence diagrams model its dynamic behavior. They illustrate how different components collaborate over time through a sequence of messages to accomplish a task. The following section details this interaction for the critical process of running a pipeline.

Figure 3.7 models the end-to-end flow of a user-triggered pipeline execution. This sequence details the chronological collaboration between the frontend, backend, and external services to fetch, execute, and report on a pipeline's logic.

The process follows these key steps:

1. **Initiation:** A User initiates a pipeline run via the User Interface.
2. **Execution Logging:** The API Endpoint receives the request and creates a new PipelineRun record in the Database to log the execution's start time and initial status.
3. **Code Retrieval:** The Pipeline Manager instructs the Git Manager to pull the latest version of the pipeline's code from the GitHub repository, ensuring the run uses the most up-to-date logic.
4. **Isolated Execution:** The Pipeline Manager then instructs the Docker Manager to create a new, isolated container for the run. Inside this container, the Step Runner executes the pipeline's steps sequentially.

5. **Cleanup and Finalization:** Upon successful completion, the Pipeline Manager instructs the Docker Manager to close the container. The API Endpoint then updates the PipelineRun record in the Database with the final status (e.g., success, failure) and completion time.
6. **Displaying Results:** Finally, the execution result is returned to the User Interface for display to the User.

3.5 Technology Stack and Work Environments

The selection of a robust technology stack is critical for project success. This section outlines the specific tools, frameworks, and platforms chosen to build a modern, maintainable, and efficient system, from local development to final deployment.

3.5.1 Development and Collaboration Tools

This foundational layer includes the tools for code creation, versioning, and team collaboration.

- **Code Editor:** For this project, Cursor, an AI-assisted fork of VSCode, was selected as the primary integrated development environment (IDE). Its intelligent features are leveraged to accelerate the development cycle and improve code quality.
- **Version Control:** Git is used for distributed version control, with GitHub serving as the central cloud-based repository. This combination facilitates collaborative development through features like pull requests, code reviews, and issue tracking.

3.5.2 Backend Technology

The backend is the engine of the platform, responsible for executing business logic, managing data, and serving the frontend application.

- **Framework:** FastAPI, a modern, high-performance Python web framework, was selected. Its choice is justified by its native asynchronous support (ASGI), ideal for I/O-bound

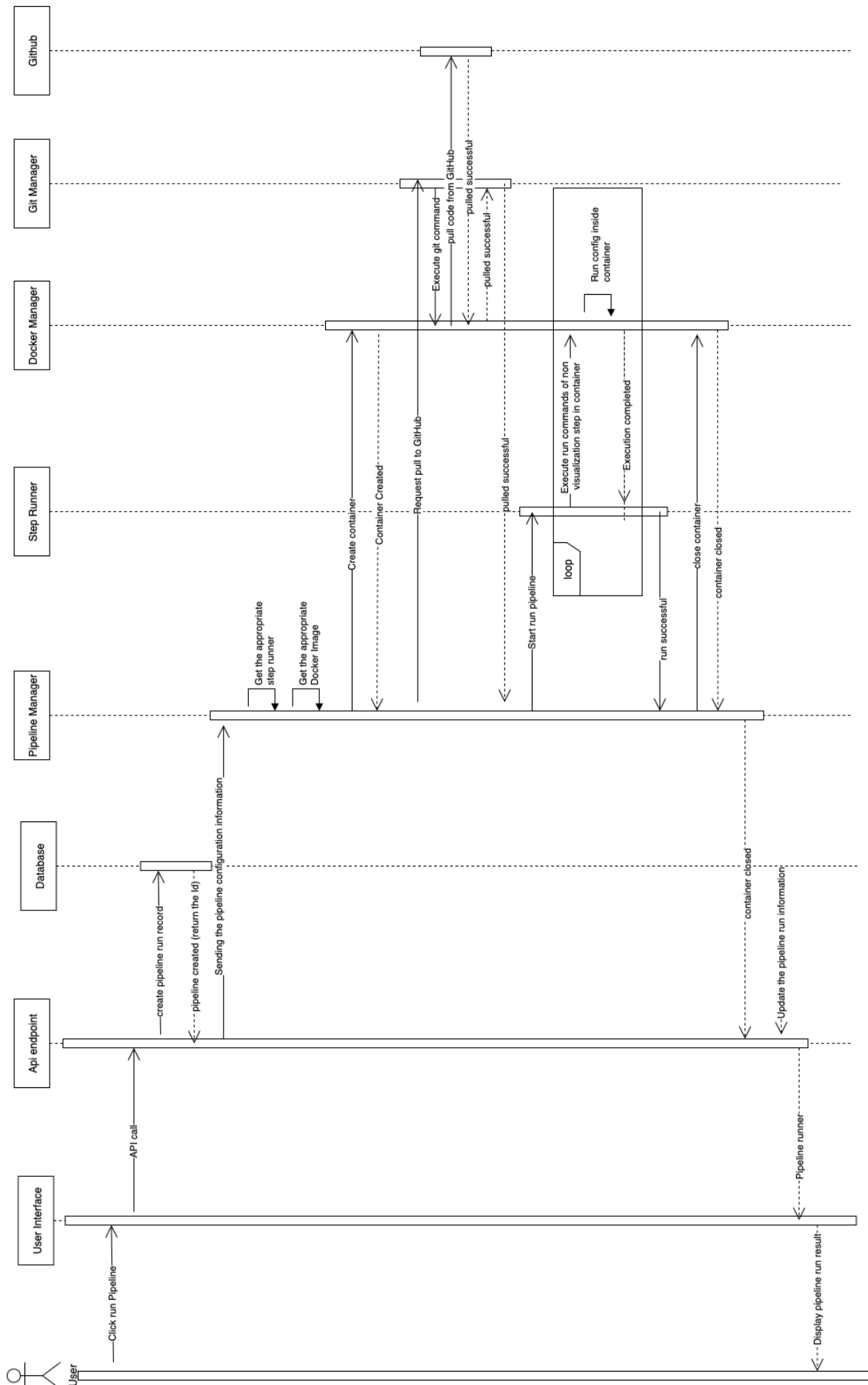


Figure 3.7: Sequence Diagram for pipeline run

tasks like database queries and API calls, its automatic generation of interactive OpenAPI documentation, and its strong performance benchmarks.

3.5.3 Frontend Technology

The frontend provides the interactive, user-facing interface for the platform.

- **Library:** React.js is the JavaScript library used to build the user interface. Its component-based architecture enables the creation of a modular and maintainable Single-Page Application (SPA), delivering a dynamic and responsive user experience.

3.5.4 Database and Storage

This component handles the persistent storage of all application data, ensuring data integrity and reliability.

- **Database Management System:** PostgreSQL is the chosen relational database, favored for its proven reliability, ACID compliance, and advanced features like robust JSONB support for storing semi-structured configuration data.

3.5.5 Containerization and Deployment

Containerization is used to package the application and its dependencies, ensuring consistency across all environments.

- **Containerization Engine:** Docker is used to containerize the frontend, backend, and database. This guarantees environmental parity between development and production, eliminating "it works on my machine" issues.
- **Orchestration:** Docker Compose is used during development to define and run the multi-container application with a single command, simplifying the local setup.

3.6 Conclusion

This chapter has established the conceptual foundation for the data pipeline platform. It has detailed the essential functional and non-functional requirements, providing a clear and comprehensive scope for the development effort. Through systematic modeling with use case diagrams, the C4 model, and UML diagrams, the system's architecture, key components, and critical interactions have been defined and validated. Finally, the selected technology stack provides a modern, robust, and efficient foundation for development. With this conceptual analysis complete, we have a validated blueprint to proceed with the detailed system design and implementation in the subsequent chapters.

Data ingestion and transformation steps

Contents

4.1	Introduction	46
4.2	Sprint 1: Data Ingestion Step Implementation	46
4.2.1	Sprint 1 Backlog	46
4.2.2	Sprint 1 Analysis	47
4.2.3	Sprint 1 Implementation	49
4.3	Sprint 2: Transformation Step Implementation	53
4.3.1	Sprint 2 Backlog	53
4.3.2	Sprint 2 Analysis	53
4.3.3	Sprint 2 Implementation	55
4.4	CONCLUSION	59

4.1 Introduction

This chapter documents the practical development work undertaken during the first two sprints of the project. It details the creation of the core data pipeline, from initial data capture to final transformation.

The chapter is divided into two main parts. First, we cover Sprint 1, which focuses on building the data ingestion system. We will walk through the process of benchmarking and selecting the best open-source tools for the job. Second, we cover Sprint 2, which addresses the data transformation layer, again detailing the tool evaluation and selection process. The goal is to provide a clear record of our technical decisions, the performance benchmarks that guided them, and the final architecture of the platform's core functionalities.

4.2 Sprint 1: Data Ingestion Step Implementation

4.2.1 Sprint 1 Backlog

The backlog for Sprint 1 focused on establishing the foundational components for data ingestion. The key deliverables that guided the sprint are summarized in Table 4.1.

Table 4.1: Sprint 1 Backlog Items

ID	Feature	User Story	Priority
1	Data Ingestion	As an administrator, I want to ingest data from diverse sources with minimal configuration so that I can rapidly set up and manage data pipelines.	High

4.2.2 Sprint 1 Analysis

This section uses diagrams to explain the key workflows designed during Sprint 1. We will focus on the user flow for configuring data ingestion and the sequence of technical operations required to create the pipeline.

4.2.2.1 Data Ingestion Flow Diagram

Figure 4.1 illustrates the high-level, step-by-step process a user follows to configure a new data ingestion pipeline. The flow ensures all necessary information is collected before the pipeline is created:

1. **Data Source Configuration:** The user first specifies the source type (e.g., PostgreSQL), credentials, and other connection details.
2. **Destination Configuration:** Next, the user defines the destination where the data will be loaded, including its type and credentials.
3. **Table and Column Selection:** Finally, the user is presented with an interface to select the specific tables and columns they wish to ingest, ensuring only relevant data is processed.

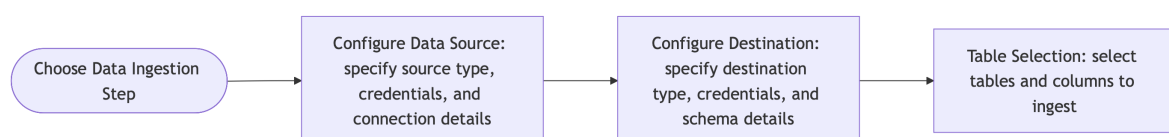


Figure 4.1: Data Ingestion Flow Diagram

4.2.2.2 Sequence Diagram for Creating an Ingestion Pipeline

The sequence diagram in Figure 4.2 illustrates the detailed technical interactions between system components when a user creates a pipeline with a data ingestion step. This process is orchestrated by the central **Pipeline Manager**.

1. **Resource Provisioning:** The Pipeline Manager requests a step runner (the executable logic) and a Docker image from the Docker Manager to create an isolated environment.
2. **Container Creation:** The Docker Manager provisions a new container for the task and confirms its creation.
3. **Pipeline Configuration:** The Pipeline Manager instructs the step runner (inside the container) to execute the ingestion step, applying the source and destination configurations.
4. **Version Control:** Once configured, the Pipeline Manager directs the **Git Manager** to push the pipeline's code to the GitHub repository, creating a permanent, versioned record.
5. **Cleanup:** After GitHub confirms the push, the Pipeline Manager instructs the Docker Manager to close the temporary container, freeing up system resources.

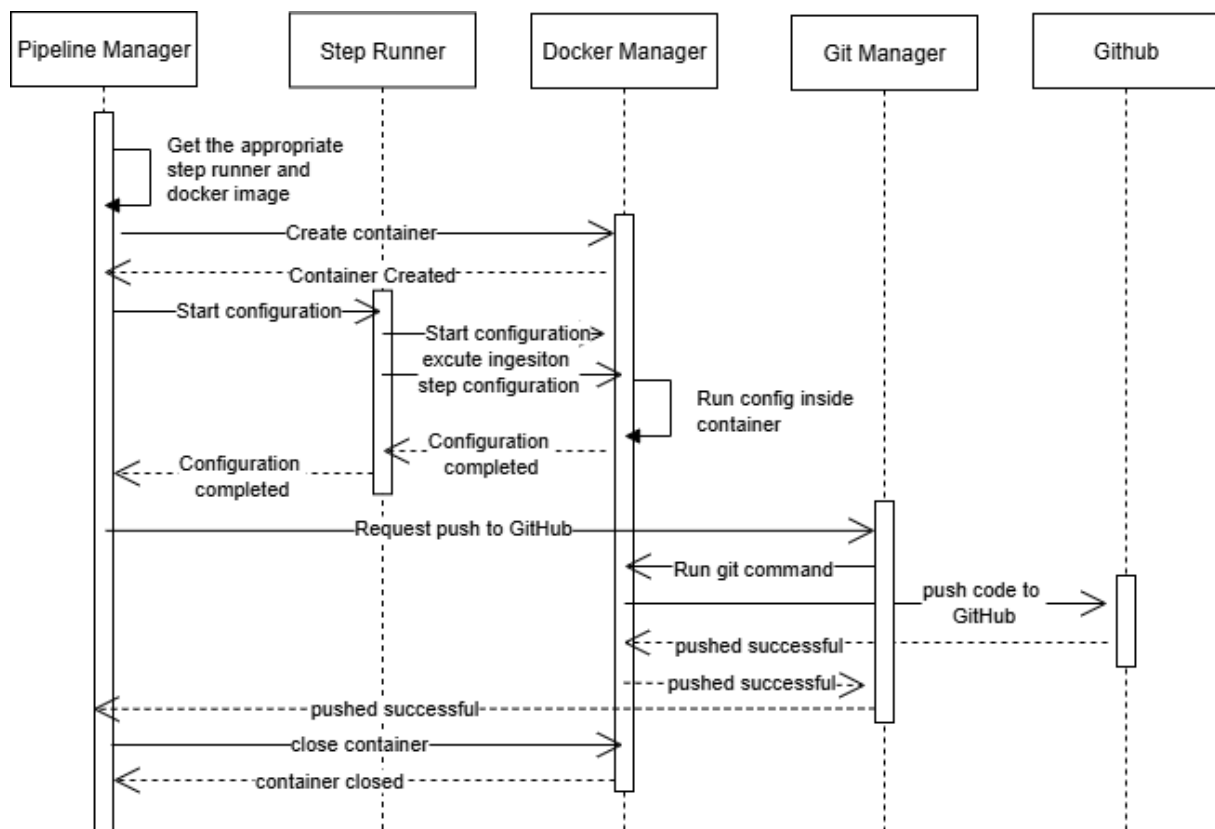


Figure 4.2: Sequence Diagram: Creating a Pipeline with a Data Ingestion Step

4.2.3 Sprint 1 Implementation

4.2.3.1 Ingestion Tool Benchmark

To select the most suitable data ingestion tool, we conducted a benchmark of four prominent open-source tools: **dlt (data load tool)**, **Airbyte**, **Apache NiFi**, and **Meltano**.

Tool Presentations

dlt (data load tool) dlt is a Python library designed for developers to build data pipelines directly in their code. It simplifies loading data from various sources into well-structured destinations. As shown by its logo in Figure 4.3, its key features include automatic schema management and a developer-centric design.



Figure 4.3: dlt (data load tool) Logo

Airbyte Airbyte is a full-featured data integration platform focused on the ELT paradigm. Its main strength is a large catalog of pre-built connectors. The Airbyte logo is shown in Figure 4.4. It offers a user-friendly web interface and ensures that connectors run in isolated Docker containers for scalability.



Figure 4.4: Airbyte Logo

Apache NiFi Apache NiFi is a system designed to automate data flow between different software systems using a visual, drag-and-drop interface. Its logo is displayed in Figure 4.5. NiFi excels at managing real-time data streams and provides detailed data tracking (provenance) from start to finish.



Figure 4.5: Apache NiFi Logo

Meltano Meltano is a DataOps platform designed to manage the entire data lifecycle by orchestrating other best-in-class open-source tools. As illustrated by its architecture in Figure 4.6, it uses the Singer standard to connect data extraction scripts ("taps") with data loading scripts ("targets"). This gives it access to hundreds of community-built connectors and allows entire data pipelines to be defined as code and version-controlled in Git.

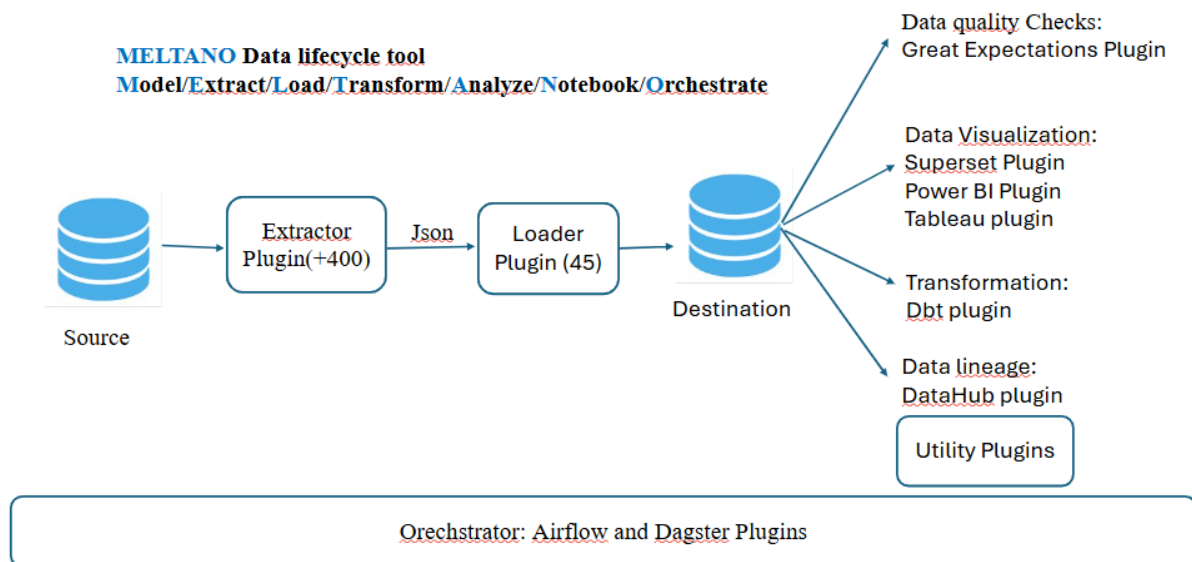


Figure 4.6: Meltano Architecture

Evaluation Criteria and Feature Comparison

To objectively assess the tools, we used five key criteria aligned with our project's needs. Table 4.2 summarizes the results.

This comparison showed that **dlt** and **Meltano** met all criteria, while Airbyte lacked parallel execution and NiFi lacked API control and schema change detection.

Table 4.2: Ingestion Tool Comparison by Feature

Criteria	dlt	Airbyte	Apache NiFi	Meltano
Parallel Execution	Yes	No	Yes	Yes
Structured/Semi-Structured	Yes	Yes	Yes	Yes
Code/API Integration	Yes	Yes	No	Yes
Schema Change Detection	Yes	Yes	No	Yes
Third-Party Integration	Yes	Yes	No	Yes

Performance Benchmark

We then conducted a performance test, inserting a growing number of rows into a PostgreSQL database. Table 4.3 shows the initial results.

Table 4.3: Ingestion Tool Initial Performance Benchmark (Time in Seconds)

Tool	100 rows	1k rows	10k rows	100k rows	1m rows
Airbyte	8s	10s	12s	18s	142s
Apache NiFi	0.36s	0.4s	0.45s	1.7s	9.2s
dlt	1s	1.3s	1.7s	2.15s	15.6s

To assess scalability, we ran an optimized test with a parallelized dlt configuration against NiFi for larger volumes. The results in Table 4.4 show dlt outperforming NiFi significantly at scale.

Table 4.4: Parallelized dlt vs. Apache NiFi Performance (Time in Seconds)

Tool	500k rows	1m rows	5m rows
Apache NiFi	5s	9.2s	44.5s
dlt (parallelized)	4.12s	5.29s	24.73s

Decision: A Hybrid Ingestion Strategy

Based on the evaluation, we adopted a two-part hybrid strategy:

1. **dlt as the Core Ingestion Engine:** Chosen for its superior performance at scale, flexibility through code, and automatic schema change detection.
2. **Meltano as the Orchestration Platform:** Chosen for its DataOps capabilities, allowing us to manage the entire data lifecycle (including dlt, dbt, etc.) as code within a single, version-controlled framework.

4.2.3.2 Ingestion Tool Implementation

Our platform's backend was built to configure and run pipelines using either dlt or Meltano, providing a unified user interface for both.

dlt and Meltano Implementation.

For both tools, the implementation follows a simple workflow. The user provides database connection details (source and destination) through the application's interface. The backend then uses this information to automatically generate the necessary configuration files for the selected tool (either dlt or Meltano) and sets up the pipeline. the supported database are Postgresql and MongoDB.

Connection Testing and Data Selection.

Before creating a pipeline, the system tests the user-provided database connections. If a connection fails, an error message guides the user to fix the settings. If successful, the application queries the source database and displays all available tables and columns. As shown in Figure 4.7, this interface allows users to browse and select the exact data they need, giving them full control and preventing the transfer of unnecessary information.

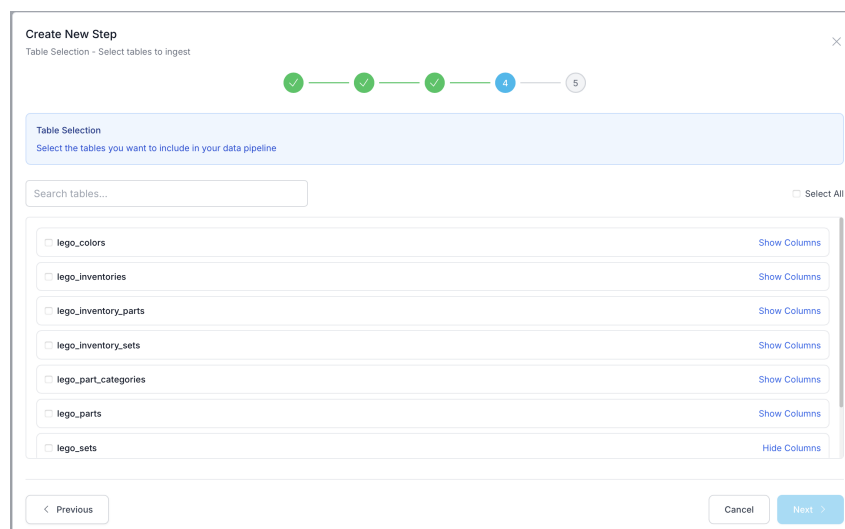


Figure 4.7: Table and Column Selection Interface

4.3 Sprint 2: Transformation Step Implementation

4.3.1 Sprint 2 Backlog

The second sprint focused on building the data transformation capabilities of the platform. The main goals for this sprint are outlined in Table ??.

Table 4.5: Sprint 2 Backlog Items

ID	Feature	User Story	Priority
1	Transformation Configuration	As an administrator, I want to easily implement and configure new data transformation steps so that I can quickly adapt pipelines to changing data requirements without developer assistance.	High

4.3.2 Sprint 2 Analysis

This section outlines the workflows designed for the data transformation step.

4.3.2.1 Data Transformation Flow Diagram

As shown in Figure 4.8, the data transformation flow begins by checking if a previous step (like ingestion) exists. If so, it automatically uses the destination database from that step as its source. If not, the user must manually configure the database connection. After establishing the source, the user proceeds to configure the specific transformation logic.

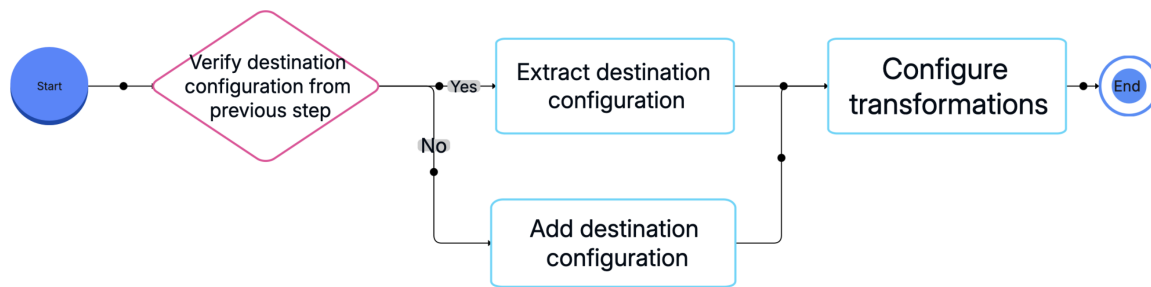


Figure 4.8: Data Transformation Flow Diagram

4.3.2.2 Sequence Diagram for Creating a Transformation Pipeline

The sequence diagram in Figure 4.9 shows the technical steps for creating a pipeline with a transformation step. The flow is nearly identical to the ingestion step, with the central **Pipeline Manager** orchestrating the process. It provisions a Docker container, executes the step runner with the appropriate transformation configurations, pushes the resulting code to GitHub, and cleans up the container afterward.

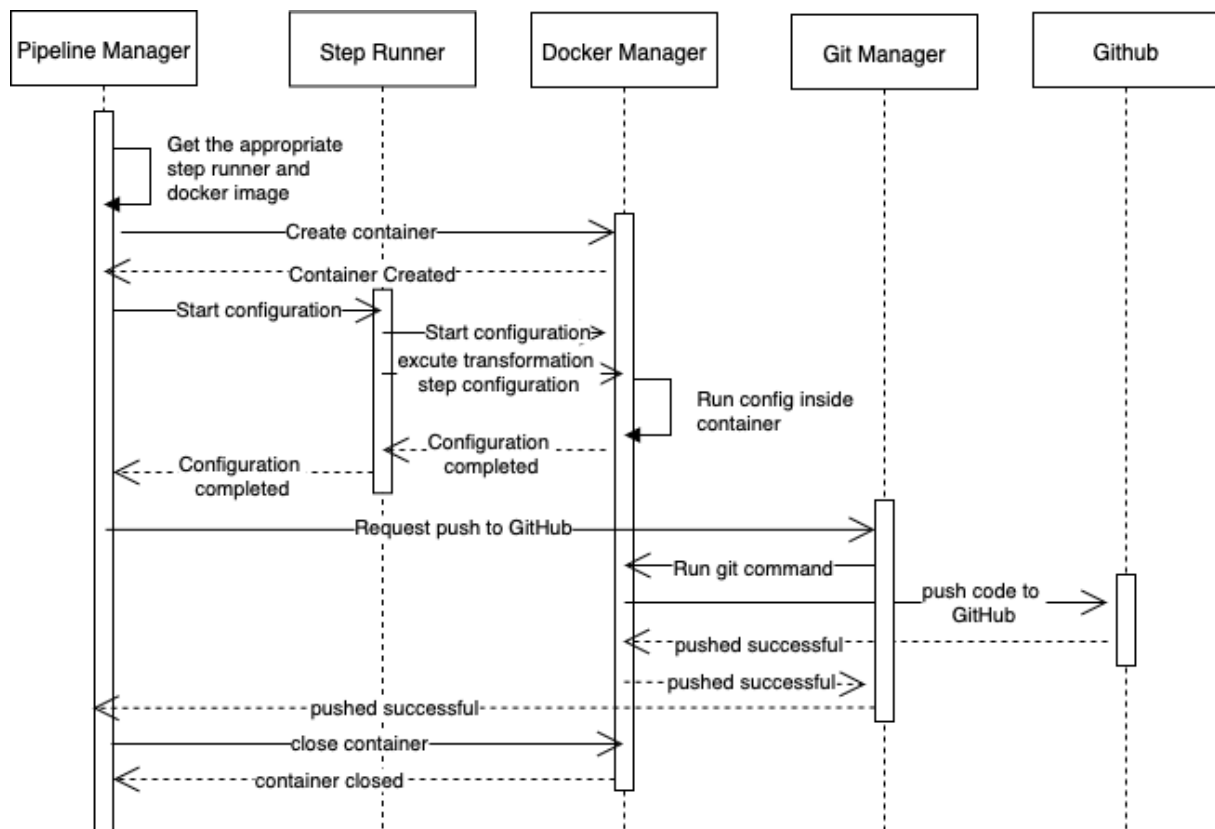


Figure 4.9: Sequence Diagram: Creating a Pipeline with a Data Transformation Step

4.3.3 Sprint 2 Implementation

4.3.3.1 Transformation Tool Benchmark

For the transformation layer, we benchmarked two leading open-source tools: **SQLMesh** and **dbt (data build tool)**.

Tools Presentation

In the landscape of modern data engineering, tools for managing transformations and data models are essential for ensuring accuracy, scalability, and maintainability. This section presents two notable frameworks—SQLMesh and dbt—that address these needs with different philosophies and technical approaches.

SQLMesh SQLMesh is a modern, open-source data transformation framework designed with a strong emphasis on reliability, efficiency, and semantic understanding of data models. By interpreting data pipelines at a conceptual level rather than as isolated SQL scripts, SQLMesh enables advanced capabilities such as:

1. **Automatic Data Validation:** Detecting inconsistencies and anomalies before they propagate downstream.
2. **Virtual Data Environments:** Allowing safe, isolated testing without impacting production datasets.
3. **Incremental Computation:** Reducing processing time by intelligently identifying and updating only the necessary data slices.

Its modular design supports integration with a variety of data warehouses and orchestration tools, making it a flexible choice for organizations seeking a modern alternative to traditional ETL workflows. The SQLMesh logo is shown in Figure 4.10.



Figure 4.10: SQLMesh Logo

dbt (data build tool) dbt is one of the most widely adopted frameworks for analytics engineering, known for transforming raw data in warehouses into clean, analytics-ready datasets using straightforward SQL. It operates on the principle of transform, don't extract or load, leaving ingestion and storage responsibilities to other systems. Key features include:

1. **Jinja-based SQL Templating:** Enabling reusable, parameterized SQL code to reduce duplication.
2. **Automated Documentation:** Providing clear visibility into dependencies and transformation flows.
3. **Testing Framework:** Offering built-in tests for schema constraints, data freshness, and business logic validation.

dbt's simplicity, combined with its integration into CI/CD pipelines, has made it a core component of modern data teams. The dbt logo is shown in Figure 4.11.



Figure 4.11: dbt Logo

Evaluation Criteria and Feature Comparison

We compared the tools based on features essential for modern transformation workflows. As Table 4.6 shows, while both tools offer a CLI and testing, SQLMesh provides advanced features like state management and virtual data environments that dbt lacks.

Table 4.6: Transformation Tool Comparison by Feature

Criteria	SQLMesh	dbt
State Management	Yes	No
Execution Efficiency	Yes	No
Virtual Data Environments	Yes	No
Command Line Interface (CLI)	Yes	Yes
Testing & Validation	Yes	Yes

Performance Benchmark

We performed a benchmark to measure transformation execution time in PostgreSQL. The results, presented in Table 4.7, show that SQLMesh consistently and significantly outperforms dbt Core across all data volumes.

Table 4.7: Transformation Performance Comparison (Execution Time in Seconds)

Tool	10K rows	100K rows	1M rows
dbt Core	12.4s	58.2s	423.7s
SQLMesh	6.8s	24.1s	146.3s

Decision: A Hybrid Transformation Strategy

Similar to ingestion, we adopted a hybrid approach orchestrated by Meltano.

1. **SQLMesh as the Primary Solution:** Chosen for its superior performance (up to 2.9x faster than dbt), advanced incremental processing, and safer development lifecycle with virtual data environments.
2. **dbt Core as a Complementary Solution:** Integrated as a Meltano plugin to leverage its vast ecosystem of packages and community support for specialized use cases.

This strategy allows us to use the best tool for the job while managing everything within Meltano's unified DataOps framework.

4.3.3.2 Transformation Tool Implementation

The transformation layer in our system supports both SQLMesh and dbt as interchangeable backends, offering flexibility for different user preferences and project requirements. These backends are selectable and configurable directly through our application's interface, enabling a smooth setup process without requiring command-line intervention.

The configuration workflow begins with setting up the destination connection—the target data warehouse or database where transformed data will be stored. The application first checks whether a valid connection was already established in a previous pipeline stage (e.g., during the ingestion phase).

1. **If a connection is detected:** If a connection is detected: The system automatically reuses it, ensuring consistency and reducing repetitive configuration.
2. **If no prior connection is found:** The user is prompted to enter the database credentials manually, specifying parameters such as host, port, database name, username, and password.

Once these details are entered, the system executes a connection test to verify that the provided credentials and network settings are correct. This validation step prevents misconfigurations from propagating downstream. Only after a successful test can the user proceed to define transformation logic—whether that logic is written as SQLMesh models or dbt models.

When using dbt, the system leverages Meltano as the orchestration layer, where dbt is installed and managed as a plugin. This approach centralizes project configuration, simplifies dependency management, and enables seamless integration with other Meltano-managed tools.

Figure 4.12 illustrates the user interface for configuring a database connection, highlighting the fields required for successful integration.

The screenshot shows a 'Create New Step' dialog box with a progress indicator at the top showing four steps: 1 (green checkmark), 2 (blue circle), 3 (grey circle), and 4 (grey circle). The main content area is titled 'Transformation Utility Selection' and includes a subtitle 'Using Sqmesh for transformation. Select the utility plugin for your transformation needs.' Below this is a 'Select Utility Plugin' dropdown menu with 'postgres' selected. Under the 'Database Connection' section, there are two radio buttons: 'Use existing connection from ingestion step' (selected) and 'Configure new connection'. A green box titled 'Using Existing Configuration' displays the following details: Source: Ingestion Step, Host: postgres_destination, Port: 5432, Database: mydatabase, User: user, Schema: public, and Ingestion Tool: dlt. At the bottom, there is a 'Connection Configuration' section with a subtitle 'Using existing connection configuration from the ingestion step.' and a 'Previous' button on the left and 'Cancel' and 'Next' buttons on the right.

Figure 4.12: Database Connection Configuration Interface

4.4 CONCLUSION

This chapter documented the implementation of the platform’s core data processing engine across two foundational sprints. By conducting rigorous, evidence-based benchmarks of leading open-source tools, we made strategic architectural decisions to build a robust, scalable, and modern data pipeline solution.

For data ingestion, we adopted a hybrid strategy, selecting dlt for its high-performance, code-centric flexibility and Meltano as the overarching DataOps framework to ensure version-controlled, reproducible workflows. For data transformation, SQLMesh was chosen as the primary engine due to its demonstrably superior performance and advanced features like state management and virtual data environments, which are critical for reliable and efficient data modeling. By integrating these best-in-class tools, we have established a powerful and maintainable foundation for the near-real-time data platform.

Advanced Transformation Types and Visualization Steps

Contents

5.1	Introduction	61
5.2	Sprint 3: Implementation of Multiple Transformation Types	61
5.2.1	Sprint 3 Backlog	61
5.2.2	Sprint 3 Analysis	62
5.2.3	Sprint 3 Implementation	65
5.3	Sprint 4: Visualization Step Implementation	71
5.3.1	Sprint 4 Backlog	71
5.3.2	Sprint 4 Analysis	71
5.3.3	Sprint 4 Implementation	73
5.4	Conclusion	76

5.1 Introduction

This chapter presents the implementation of core data transformation and visualization capabilities in our pipeline platform. Sprint 3 focuses on developing three transformation approaches: simple column operations for basic data cleaning, complex joins for multi-table analysis, and natural language-powered agentic transformations.

Sprint 4 builds upon this foundation by implementing the visualization layer, integrating dashboard creation and management through Apache Superset. The implementation covers automated dashboard generation, secure access controls, and seamless connection with transformed data sources.

5.2 Sprint 3: Implementation of Multiple Transformation Types

5.2.1 Sprint 3 Backlog

The Sprint 3 backlog focused on establishing the data transformation type pipeline. The key deliverables are summarized in Table 5.1:

Table 5.1: Sprint 3 Backlog Items

ID	Feature	User Story	Priority
1	Transformation Management	As an administrator, I want to manage and configure all data transformation pipelines so that I can ensure data quality and flow across the system.	High
2	Data Integration Control	As an administrator, I want to define and control multi-table join rules so that I can securely integrate data from various tables for analysis.	High

Table 5.1 continued from previous page

ID	Feature	User Story	Priority
3	Automated Transformation Deployment	As an administrator, I want to enable and configure the natural language-to-SQL transformation feature so that I can empower business users to perform data transformations without writing code.	Medium

5.2.2 Sprint 3 Analysis

In this section, we want to distill the Sprint 3 actions and objectives into a list of diagrams to build a mental model around the workflows involved. We will focus on the data transformation flow and the sequence of operations for pipeline creation.

5.2.2.1 Data Transformation Type Flow

1. Complex Transformation

1. **Start** – The process begins after the source is confirmed.
2. **Choose Main Table** – The user selects the primary table to be used as the base for the join.
3. **Choose Tables and Join Type** – The user selects one or more additional tables and specifies the type of join (e.g., INNER, LEFT, RIGHT, FULL).
4. **Choose Selected Columns and Join Condition** – The user selects which columns to keep and defines the join condition(s).
5. **End** – The join transformation configuration is complete.

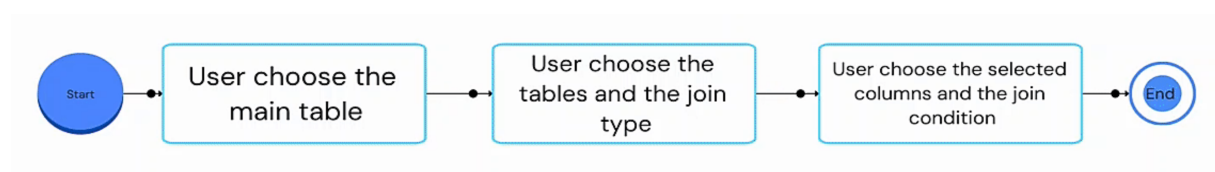


Figure 5.1: Join Transformation Workflow

2. Simple Transformation

1. **Start** – The process begins after the source is confirmed.
2. **Retrieve Tables and Columns** – The user is presented with all tables, their columns, and available simple transformations (e.g., rename column, change data type).
3. **Choose Supported Function** – The user selects one of the available transformation functions to apply.
4. **End** – The simple transformation configuration is complete.

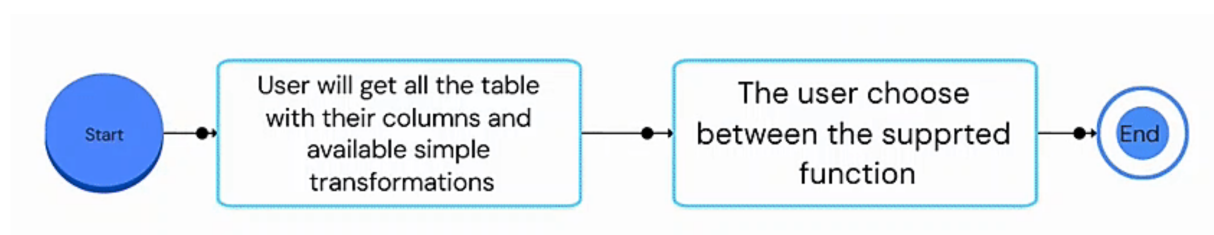


Figure 5.2: Simple Transformation Workflow

3. Agentic Transformation

In this transformation type, the user provides their intent in natural language, describing the desired data manipulation or query outcome. The system then automatically generates an equivalent SQL query based on the provided intent. This generated SQL query is displayed to the user in a modal interface for review, allowing the user to validate or modify it before final execution. This approach combines the convenience of intent-based input with the safety of user verification.

5.2.2.2 Sequence Diagram for Creating dbt or SQLMesh Model

This sequence diagram illustrates the detailed interactions between system components when setting up a **data transformation** step within a data pipeline. The process begins *after* the initial database records for the pipeline, pipeline step, and pipeline step configuration have been successfully created. The orchestration of this process is handled by the **Pipeline Manager**, which serves as the central control unit.

1. **Container Provisioning:** The Pipeline Manager initiates the process by requesting the **Docker Manager** to create a new container. This container provides an isolated execution environment for running the transformation tool.
2. **Transformation Step Configuration:** Once the container is ready, the Docker Manager configures the transformation step and initiates the selected **Transformation Tool** (e.g., dbt or SQLMesh).
3. **Tool Initialization:** The Transformation Tool is initialized and sets up the project structure, including creating folders such as `models/staging` and `models/core`.
4. **Database Connection Setup:** The Transformation Tool configures the database connections required for executing transformation models.
5. **Model Configuration:** The Transformation Tool communicates with the **Transformation Manager** to handle the configuration of transformation models, ensuring that parameters, dependencies, and execution settings are properly defined.
6. **Model and YAML File Generation:** Once configured, the Transformation Tool generates the SQL files for the models along with associated YAML files. These outputs are then relayed back to the Docker Manager and Pipeline Manager for integration into the pipeline.

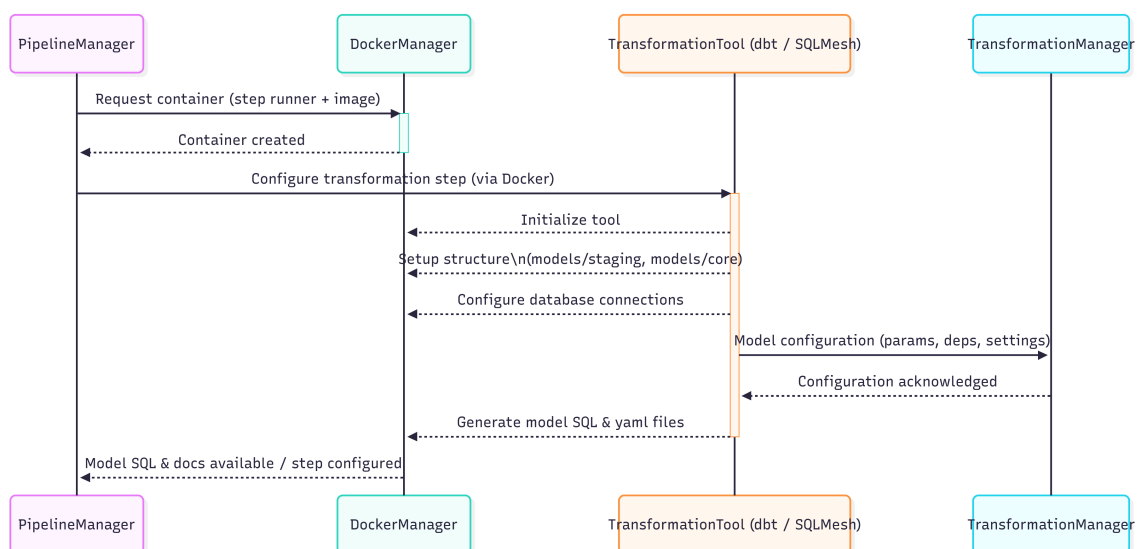


Figure 5.3: Sequence Diagram: Creating Pipeline with Data Ingestion Step

5.2.3 Sprint 3 Implementation

5.2.3.1 Simple Transformation

The platform supports a comprehensive suite of simple column transformations to handle common data cleaning and preparation tasks:

String Transformations:

- **Case Conversion:** UPPER, LOWER

Null Value Handling:

- **Statistical Imputation:** Replace nulls with MEAN, MEDIAN, MODE values
- **Boundary Imputation:** Use MIN or MAX values for null replacement

Date and Time Transformations:

- **Component Extraction:** Extract DAY, MONTH, YEAR components

The user interface presents available tables and columns in an intuitive selection interface, allowing users to choose desired transformation functions through dropdown menus and configuration panels.

Backend Processing:

In the backend, these transformations are implemented using pre-defined **macro files**. Each macro contains the reusable SQL or transformation logic for a specific operation (e.g., converting text to upper case, replacing nulls, or extracting a date component). When a user selects a transformation in the interface, the platform does not rewrite the logic from scratch. Instead, it **injects the user-provided parameters** (such as the target column, replacement value type, or date part) into the corresponding macro. This approach ensures:

- **Consistency** — the same logic is applied regardless of where or how it is used.
- **Maintainability** — updates to transformation logic require changes only in the macro file.
- **Flexibility** — parameters allow macros to adapt to different tables, columns, or transformation settings without duplicating code.

5.2.3.2 Complex Transformation

For advanced analytical scenarios, the system provides comprehensive support for complex multi-table join transformations:

Join Operation Support:

- **INNER JOIN:** Return records with matching values in both tables
- **LEFT JOIN:** Return all records from left table and matched records from right table
- **RIGHT JOIN:** Return all records from right table and matched records from left table
- **FULL OUTER JOIN:** Return all records when there's a match in either table
- **CROSS JOIN:** Return Cartesian product of both tables

Multi-Table Join Capabilities:

- Support for joining multiple tables in a single operation
- Complex join condition specification with multiple column matching

The user workflow for complex transformations follows a structured approach:

1. **Primary Table Selection:** Choose the main table as the join foundation
2. **Join Table Selection:** Select additional tables for join operations
3. **Join Type Configuration:** Specify the appropriate join type for each table relationship
4. **Join Condition Definition:** Define column-based join conditions and relationships
5. **Output Column Selection:** Choose which columns to include in the final result set

Backend Processing:

Once the user completes the configuration in the interface, the backend processes the provided selections and automatically **generates the SQL file** required for the join transformation. This generation process involves:

- Parsing the user-defined join types, conditions, and selected columns.
- Mapping each join operation to the corresponding SQL syntax.
- Structuring the final query to include all necessary joins and selected fields.
- Saving this SQL logic into a dedicated transformation file (e.g., a .sql file in the transformation project structure) that can be executed by the chosen transformation tool (such as dbt or SQLMesh).

This automated SQL generation ensures that complex joins are consistently implemented, optimized, and ready for execution without requiring manual query writing from the user.

5.2.3.3 Agentic Transformation

This transformation type leverages an innovative **agentic approach** powered by a **Large Language Model (LLM)** to dynamically generate SQL queries from natural language prompts. This feature enables non-technical users to perform complex data operations through conversational interfaces.

Benchmark Results

Our evaluation of leading SQL generation models yielded the following key findings based on comprehensive testing:

SQL Query Generation Model Comparison										
Metric	Claude 3.7 Sonnet	Gemini 2.5 Pro	Gemini 2.0 Flash	Llama 4 Maverick	DeepSeek V3	Grok-3-Beta	Grok-3-Mini-Beta	OpenAI O3-Mini	Quasar Alpha	Optimus Alpha
Performance Metrics										
Average Score	0.660	0.880	0.717	0.565	0.617	0.747	0.645	0.635	0.820	0.830
Median Score	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Standard Deviation	0.455	0.300	0.392	0.488	0.460	0.405	0.459	0.464	0.357	0.359
Success Rate	75.0%	92.5%	92.5%	62.5%	75.0%	90.0%	72.5%	72.5%	87.5%	87.5%
Efficiency & Cost										
Avg. Execution Time (ms)	2,003	2,478	1,296	1,986	26,892	1,707	1,593	8,854	1,514	1,859
Input Cost (\$/M tokens)	\$3.00	\$1.25	\$0.10	\$0.19	\$0.27	\$3.00	\$0.30	\$1.10	\$0.00	\$0.00
Output Cost (\$/M tokens)	\$15.00	\$10.00	\$0.40	\$0.85	\$1.10	\$15.00	\$0.50	\$4.40	\$0.00	\$0.00
Score Distribution										
0.0-0.2	32.5%	10.0%	22.5%	42.5%	37.5%	25.0%	35.0%	37.5%	17.5%	17.5%
0.3-0.5	2.5%	2.5%	7.5%	0.0%	2.5%	0.0%	0.0%	0.0%	0.0%	0.0%
0.6-0.7	0.0%	0.0%	2.5%	2.5%	0.0%	5.0%	5.0%	0.0%	2.5%	0.0%
0.8-0.9	7.5%	5.0%	12.5%	2.5%	7.5%	2.5%	0.0%	5.0%	7.5%	2.5%
1.0 (Perfect Score)	57.5%	82.5%	55.0%	52.5%	52.5%	67.5%	60.0%	57.5%	72.5%	80.0%

Figure 5.4: Complete SQL Query Generation Model Comparison Metrics (Source [10])

The benchmarking data reveals several critical insights:

- **Performance Tradeoffs:** While Gemini 2.5 Pro achieved the highest average score (0.890), Gemini 2.0 Flash delivered identical success rates (92.5%) with nearly 2x faster execution (1,296ms vs 2,478ms)
- **Cost Efficiency:** Gemini 2.0 Flash demonstrated superior cost-effectiveness at \$0.10 per million input tokens, making it:
 - 12.5x cheaper than Gemini 2.5 Pro (\$1.25/M)
 - 30x cheaper than Claude 3.7 (\$3.00/M)
- **Context Capacity:** Both Gemini models offer industry-leading 1 million token context windows, while Claude 3.7 provides 200K token capacity
- **Consistency:** All models showed perfect median scores (1.000), indicating reliable performance on standard queries
- **Error Distribution:** Claude 3.7 showed more frequent low scores (32.5% in 0.0-0.2 range) compared to Gemini variants (10% and 22.5% respectively)

Based on these findings [10], we recommend Gemini 2.0 Flash for most enterprise SQL generation needs due to its unique combination of:

- Market-leading 1M token context capacity
- Best-in-class cost-performance ratio (\$0.10/M tokens)
- 1.3s average response times
- 92.5% success rate (tied with Gemini 2.5 Pro)

LLM Integration with n8n

For implementing the agentic transformation workflow, we utilized **n8n**, an open-source workflow automation platform that enables visual workflow creation and execution. n8n is a low-code/no-code automation tool that allows users to connect different services and APIs through a node-based interface, making it ideal for building complex data processing pipelines [6].

Why n8n for LLM Integration

We chose n8n for our LLM integration for several key reasons:

- **Visual Workflow Design:** n8n's intuitive drag-and-drop interface allows for easy visualization and modification of the transformation pipeline
- **Extensible Architecture:** Built-in support for HTTP requests and webhooks enables seamless integration with various LLM APIs
- **Error Handling:** Robust error handling and retry mechanisms ensure reliable operation in production environments
- **Scalability:** Support for distributed execution and queue management for handling high-volume data processing
- **Open Source:** Cost-effective solution with full control over deployment and customization
- **Community Support:** Active community with 200+ pre-built nodes and integrations

- Syntax validation
 - Performance considerations
6. **Query Validation:** Before execution, queries are validated using SQLFluff [7] for syntax correctness and potential security vulnerabilities.
 7. **Query Return and Response:** Valid queries are returned to the user for review.
 8. **Feedback Loop:** The system implements continuous learning through query success/failure tracking to improve future SQL generation accuracy.

5.3 Sprint 4: Visualization Step Implementation

5.3.1 Sprint 4 Backlog

The Sprint 4 backlog focused on implementing a comprehensive data visualization and dashboard solution to complete the end-to-end data pipeline. The key deliverables are summarized in Table 5.2:

Table 5.2: Sprint 4 Backlog Items

ID	Feature	User Story	Priority
1	Dashboard Management	As a user, I want to use an integrated visualization tool to create and manage dashboards so that I can easily monitor and analyze my data.	High

5.3.2 Sprint 4 Analysis

This section explains the key workflows designed during Sprint 4, focusing on the user flow for configuring data visualization and the sequence of technical operations required to create the pipeline.

5.3.2.1 Data Visualization Flow Diagram

The high-level process for configuring a new data visualization pipeline is illustrated in Figure 5.6.

The flow ensures all necessary steps are completed before the pipeline is finalized:

1. **Start:** The process begins with the initialization of the pipeline configuration.
2. **Verify Destination Configuration:** The system checks if the destination configuration from the previous step is valid. The user can respond with:
 - **Yes:** Proceeds to extract the destination configuration
 - **No:** Redirects to add or correct the destination configuration
3. **Extract Destination Configuration:** The system retrieves the validated configuration details.
4. **End:** The process concludes once the configuration is successfully extracted or updated.

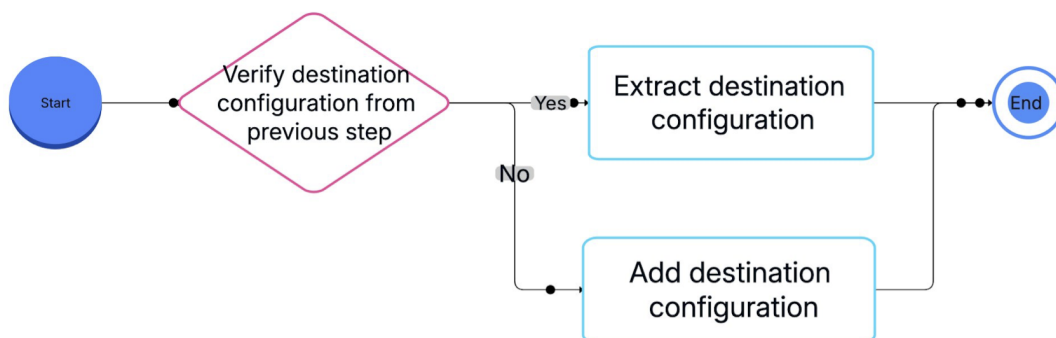


Figure 5.6: Data Visualization Configuration Flow Diagram

5.3.2.2 Sequence Diagram for Pipeline Creation

The sequence diagram in Figure 5.7 details the technical interactions between system components during pipeline creation:

1. Pipeline Manager:

- Gets the appropriate step runner and Docker image
- Creates container

- Starts configuration

2. Step Runner:

- Adds database configuration
- Creates the dashboard

3. Docker Manager:

- Sets configuration completed status
- Stops the container
- Stores and closes container

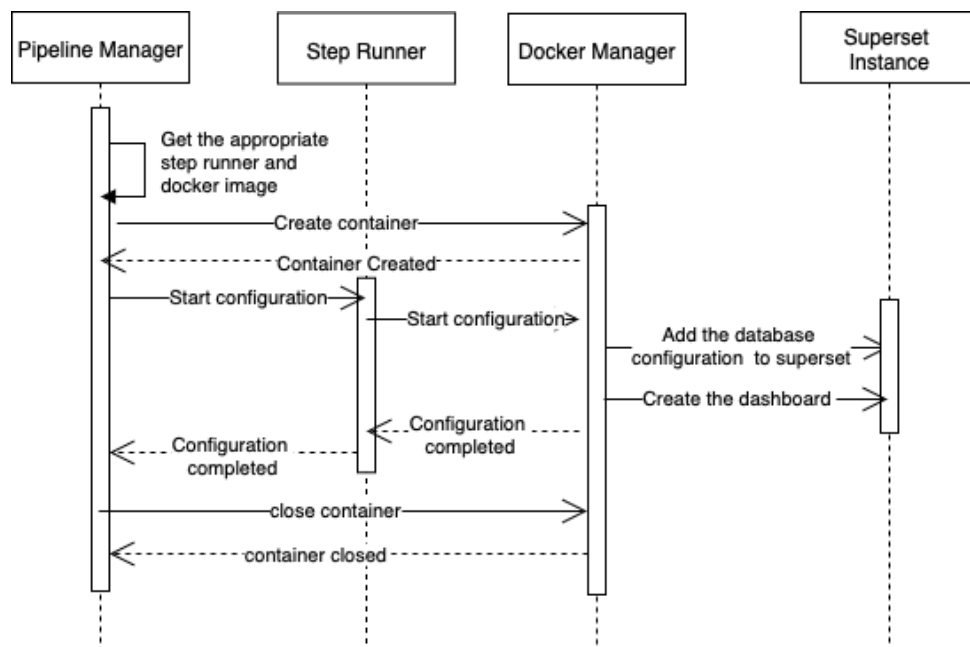


Figure 5.7: Pipeline Creation Sequence Diagram

5.3.3 Sprint 4 Implementation

5.3.3.1 Visualization Platform Selection

We selected **Apache Superset** as our visualization platform due to its comprehensive capabilities that perfectly match our technical requirements:



Figure 5.8: Apache Superset Logo

- **Rich Visualization Library:** Offers 50+ chart types including advanced visualizations like geospatial mapping, hierarchical trees, and time-series analytics, enabling users to create insightful dashboards without coding
- **Enterprise Scalability:** Proven capability to handle billion-row datasets through its intelligent query processing engine and result caching system
- **Extensive Connectivity:** Native support for all major databases and data warehouses through SQLAlchemy integration, with optimized connectors for platforms like Snowflake and BigQuery
- **Robust Security Model:** Features enterprise-grade security including:
 - Row-level security filters
 - Integration with LDAP/Active Directory
 - Granular permission controls
- **Programmable Interface:** Comprehensive REST API and Python SDK enable complete automation of dashboard creation and management
- **Active Development:** As an Apache Software Foundation project, benefits from:
 - Regular security updates
 - Continuous feature enhancements
 - Strong community support
- **Embedding Capabilities:** Unique ability to securely embed interactive visualizations in external portals and applications

The platform's technical architecture aligns perfectly with our requirements for:

- Handling complex analytical workloads
- Seamless integration with modern data stacks
- Enterprise-grade security compliance
- Scalable performance

Superset's combination of visualization power, security features, and automation capabilities makes it the ideal choice for our data platform's visualization layer.

5.3.3.2 User Management Integration

The platform implements tight integration with Superset's user management system:

- **Automatic User Provisioning:** When a new user is created in our platform, a corresponding user account is automatically created in Superset
- **Role-Based Access Control:** We implement two primary roles:
 - **Gamma:** Read-only access to assigned dashboards
 - **Admin:** Full access to all dashboards and administrative functions
- **Secure Credential Management:** User credentials are securely synchronized between systems

5.3.3.3 Dashboard Automation

The implementation includes automated dashboard creation and management:

- **Pipeline Integration:** When a visualization pipeline is created:
 - The system automatically establishes a connection to the target database.
 - Creates a new dashboard in Superset.
 - Grants access to this dashboard.

5.4 Conclusion

The completion of Sprints 3 and 4 marks significant progress in our data pipeline platform's development. Sprint 3 delivered a versatile transformation framework comprising three complementary approaches: simple column operations for basic data cleaning, complex join capabilities for multi-table analysis, and an innovative LLM-powered natural language interface that democratizes data transformation. Building on this foundation, Sprint 4 successfully integrated Apache Superset to complete the end-to-end workflow, implementing automated dashboard generation with secure role-based access control. Together, these components form a cohesive system that guides data from raw ingestion through transformation to actionable visual insights while maintaining enterprise-grade security and scalability. These sprints have established a robust foundation for the platform, enabling users to derive meaningful insights from their data through a seamless, end-to-end workflow.

Authentication, Authorization, and Change Data Capture

Contents

6.1	INTRODUCTION	78
6.2	Sprint 5: Authentication and Authorization Component . . .	78
6.2.1	Sprint 5 Backlog	78
6.2.2	Sprint 5 Analysis	78
6.2.3	Sprint 5 Implementation	81
6.3	Sprint 6: Schema Change Detection Implementation	85
6.3.1	Sprint 6 Backlog	85
6.3.2	Sprint 6 Analysis	86
6.3.3	Sprint 6 Implementation	89
6.4	Conclusion	96

6.1 INTRODUCTION

This chapter details the work completed during the fifth and sixth sprints of the project. Sprint 5 was dedicated to implementing a robust Authentication and Authorization system to secure the application using Keycloak. Sprint 6 focused on developing the core analytical engine of the project: an event-driven Schema Change Detection feature using Debezium and Kafka, which allows users to track and visualize the evolution of their database schema over time in real-time.

6.2 Sprint 5: Authentication and Authorization Component

6.2.1 Sprint 5 Backlog

The Sprint 5 backlog focused on establishing the authentication and authorization components. The key deliverables are summarized in Table 6.1:

Table 6.1: Sprint 5 Backlog Items

ID	Feature	User Story	Priority
1	Secure Authentication	As a user, I want to log in securely with my credentials so that my account is protected from unauthorized access.	High
2	Role-Based Authorization	As a system administrator, I want to assign users either the <i>Admin</i> or <i>User</i> role so that access to system features is restricted according to their role.	High

6.2.2 Sprint 5 Analysis

This sprint focused on integrating Keycloak, an open-source Identity and Access Management (IAM) solution, into our FastAPI application to handle authentication and authorization. Using

an external IAM system allowed us to avoid building custom, and potentially vulnerable, login and role-management logic. It also provided immediate access to enterprise-level security features such as secure password hashing, session management, token issuance, and centralized user and role control.

The analysis is divided into two core components: authentication and authorization.

6.2.2.1 Authentication Component

The application relies on Keycloak, a dedicated identity and access management system, to handle user authentication securely. The process is designed to be both user-friendly and robust, leveraging the OpenID Connect (OIDC) protocol.

The authentication workflow, illustrated in Figure 6.1, is as follows:

1. An unauthenticated user opens the Power App (client UI).
2. The user enters their credentials (username and password) into the app's custom login form.
3. The client UI securely sends the credentials to the FastAPI backend's dedicated login endpoint.
4. The FastAPI backend receives the credentials and forwards them to Keycloak's token endpoint, acting on behalf of the user.
5. Keycloak validates the credentials against its user database.
6. Upon successful validation, Keycloak returns a signed JSON Web Token (JWT) and a refresh token to the FastAPI backend.
7. The backend securely responds to the client UI with the tokens or a session identifier.
8. The client UI stores the received token and attaches it to all subsequent requests to the FastAPI backend.
9. The FastAPI backend verifies the JWT's signature against Keycloak's public keys for each request to ensure its authenticity.

This approach ensures that plain-text passwords are never handled by the client-side application logic. All sensitive authentication processes are orchestrated by the secure backend server, significantly reducing the attack surface and simplifying security maintenance.

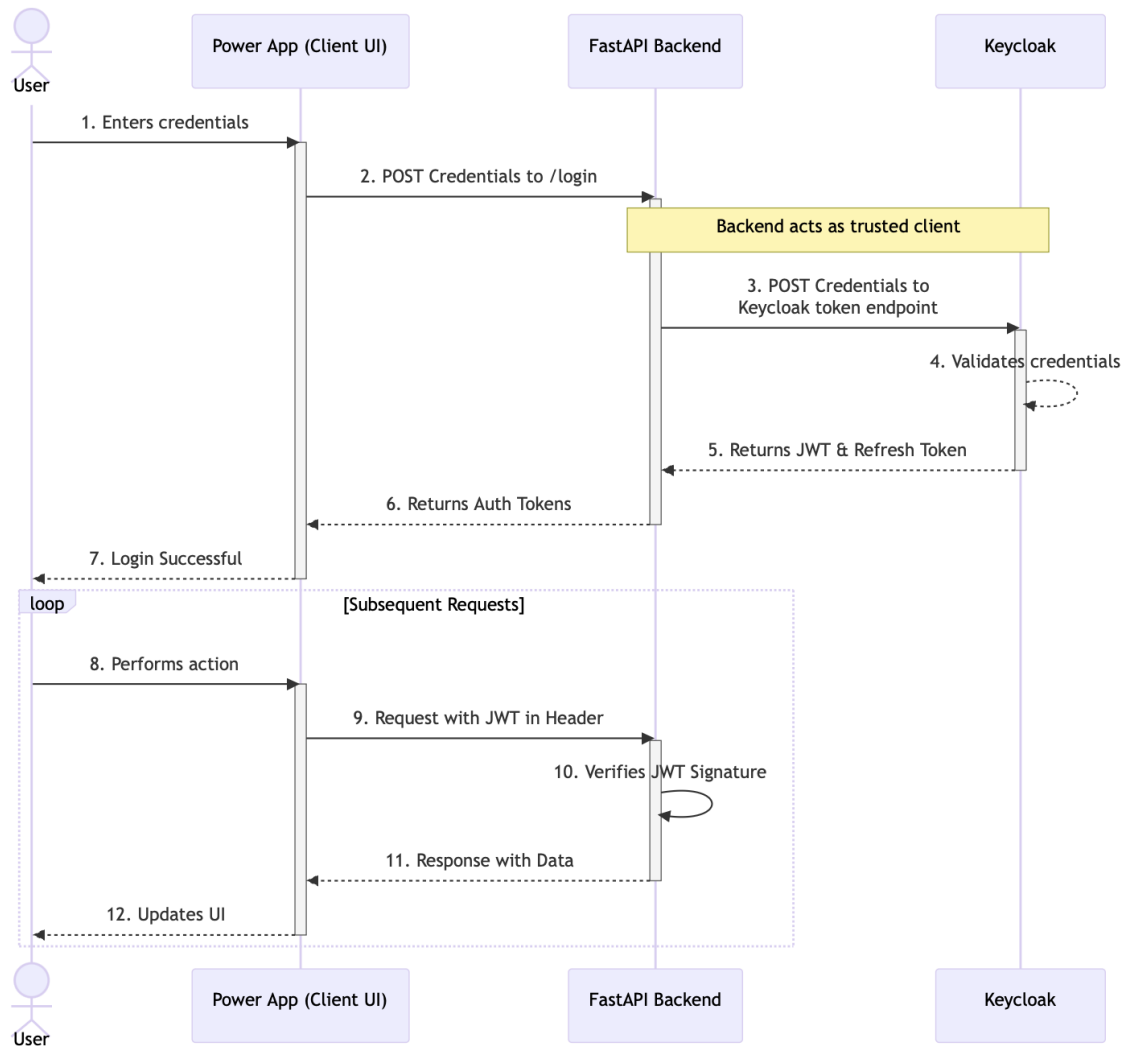


Figure 6.1: Authentication workflow with Keycloak

6.2.2.2 Authorization Component

Authorization defines the permissions granted to an authenticated user—what they are allowed to see and do within the application. We implemented a Role-Based Access Control (RBAC) model, centralizing permission management within Keycloak. Two primary roles govern access:

- **Admin:** Grants complete system control. Permissions include managing users (inviting, disabling), configuring system settings, and accessing all analytical features and data, regardless of ownership.

- **User:** Grants limited access to essential functionalities. Permissions are restricted to connecting personal databases, running analyses, and viewing own results. Access to other users' data or administrative features is prohibited.

The user's assigned roles are embedded as claims within the JWT received from Keycloak during authentication. The authorization workflow is as follows:

1. A client request, including the JWT in the Authorization header, is sent to a protected API endpoint.
2. The FastAPI backend intercepts the request, validates the JWT's signature, and decodes its payload.
3. The backend extracts the user's roles from the JWT.
4. It then checks if the required role for the requested endpoint matches the user's roles.
5. If the user has the required role (e.g., an Admin role for /api/admin/users), the request is fulfilled.
6. If the user lacks the required role, the backend rejects the request and returns a 403 Forbidden response.

This model decouples permission logic from the application code, centralizing it within Keycloak. This simplifies auditing and modifying access policies without requiring changes to the application's codebase.

6.2.3 Sprint 5 Implementation

6.2.3.1 Keycloak Definition and Rationale

For our authentication and authorization implementation, we selected **Keycloak**, an open-source Identity and Access Management (IAM) solution developed by Red Hat. Keycloak provides comprehensive authentication and authorization capabilities that would be complex and time-consuming to build from scratch.

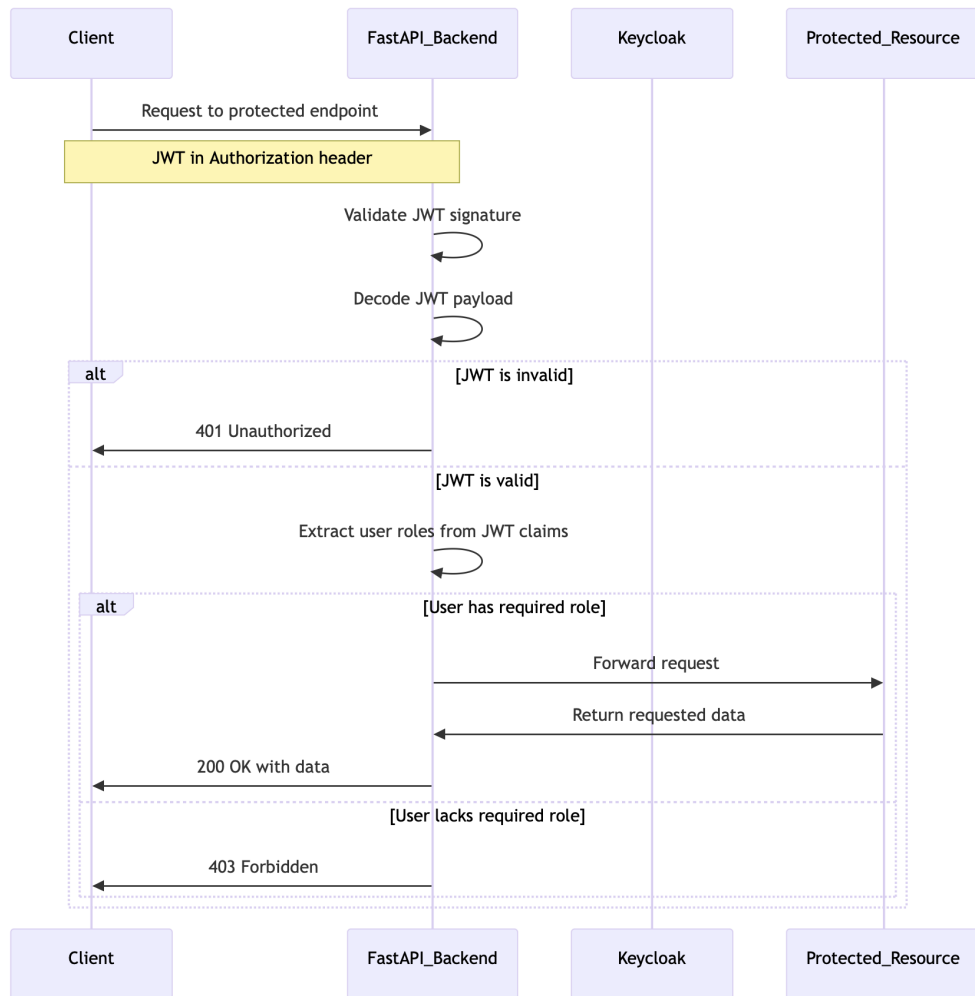


Figure 6.2: Authorization workflow with Admin and User roles

6.2.3.2 Why Keycloak?

We chose Keycloak for several compelling reasons:

- **Open Source and Enterprise-Ready:** Keycloak is a robust, open-source solution that provides enterprise-grade features without licensing costs, making it ideal for our platform.
- **Standards Compliance:** Keycloak supports industry standards including OpenID Connect (OIDC), OAuth 2.0, and SAML 2.0, ensuring interoperability and future-proofing our authentication implementation.
- **Centralized Identity Management:** It provides a single point of control for user identities, roles, and permissions across our entire application ecosystem.

- **Security Best Practices:** Keycloak incorporates security best practices by default, including secure token handling, password policies, and protection against common attacks like brute force and session fixation.
- **Extensibility:** Keycloak's architecture allows for custom extensions and integrations, providing flexibility for future requirements.
- **User Federation:** It supports connecting to existing user directories (LDAP, Active Directory) which would be essential for enterprise adoption.
- **Reduced Development Overhead:** By leveraging Keycloak, we avoided building and maintaining complex authentication and authorization logic, allowing our team to focus on core application functionality.

6.2.3.3 Authentication Implementation

The authentication implementation involved configuring both the Keycloak server and our FastAPI application:

1. Keycloak Server Setup:

- Deployed Keycloak in a Docker container for easy management and scalability
- Created a dedicated realm for our application to isolate configuration and users
- Configured a client application within Keycloak with appropriate redirect URIs
- Configured token expiration policies appropriate for our use case

2. FastAPI Backend Integration:

- Implemented OIDC authentication endpoints using the 'python-keycloak' library
- Created authentication middleware that validates JWT tokens on each request
- Implemented token refresh logic to maintain user sessions
- Set up secure cookie handling for session management

3. React Frontend Integration:

- Integrated the 'keycloak-js' library for frontend authentication flows
- Implemented login/logout components with proper state management
- Added automatic token refresh before expiration
- Created protected route components that redirect unauthenticated users

6.2.3.4 Authorization Implementation and User Capabilities

We implemented a comprehensive Role-Based Access Control (RBAC) system with two distinct user roles, each with specific capabilities:

Admin Role Capabilities:

- **User Management:** Create, update, disable, and delete user accounts
- **pipeline creation and delete and edit:** Modify application-wide settings and configurations
- **pipeline access management:** View and manage all data pipelines across all users
- **Monitoring:** Access pipeline performance metrics
- **Role Management:** Assign and modify user roles and permissions

User Role Capabilities:

- **view and run pipelines:** Create, configure, and execute their own data pipelines
- **Profile Management:** Update their personal information and preferences

6.2.3.5 Pipeline Authorization

A critical aspect of our authorization implementation was pipeline-level access control. While each pipeline has an owner, access is governed by a granular permissions system. We implemented the following authorization rules:

- **Role-Based Access:** Users can only view, modify, and execute pipelines for which they have been explicitly granted permissions.

- **Admin Override:** Admin users bypass permission checks and have full access to all pipelines in the system.

This was implemented through a combination of:

- A dedicated pipeline_user_access join table that manages user access rights.
- API endpoint authorization checks that verify the user's admin status or look up their permissions in the access table.
- Frontend UI components that conditionally render based on the user's granted permissions.

This comprehensive authorization implementation provides a secure and flexible foundation, allowing for precise access control that is centrally managed within the backend.

6.3 Sprint 6: Schema Change Detection Implementation

6.3.1 Sprint 6 Backlog

Sprint 6 focused on implementing the core analytical engine of the platform: real-time schema change detection and monitoring. The key deliverables are summarized in Table 6.2 :

Table 6.2: Sprint 6 Backlog Items

ID	Feature	User Story	Priority
1	Threshold-Based Processing	As a user, I want the schema analysis pipeline to automatically trigger after a specific number of changes or events have occurred so that I can process them efficiently in batches.	High

Table 6.2 continued from previous page

ID	Feature	User Story	Priority
2	Change Detection Engine	As a user, I want to automatically track additions, modifications, and deletions of tables and columns so that I can maintain an accurate view of my database evolution.	High
3	Historical Schema Tracking	As a data analyst, I want to view the complete history of schema changes so that I can understand how my database structure has evolved over time.	High
4	Schema Change Alerting	As a user, I want to receive email alerts when a breaking schema change happens so that I can be notified immediately and take action.	High

6.3.2 Sprint 6 Analysis

This sprint focused on designing the core analytical feature of the platform: a real-time, event-driven engine for detecting and tracking changes in database schemas. The primary goal was to move beyond simple snapshots and provide a live, historical view of a database's structural evolution with minimal performance impact on the source databases.

6.3.2.1 User Interaction and Workflow Analysis

The designed user experience for schema change tracking is a multi-step process. The workflow, depicted in Figure 6.3, begins with the user configuring a pipeline. This configuration includes the necessary credentials and permissions for the system to access the database's transaction log, along with critical parameters such as the change threshold that triggers analysis.

Once the pipeline is created, the change capture process is activated. Our solution employs a hybrid event-driven architecture where the system handles different types of changes with appropriate strategies:

- **Breaking changes** (e.g., DROP TABLE, DROP COLUMN, destructive ALTER) are processed immediately to ensure timely alerts
- **Non-breaking changes** (e.g., ADD COLUMN, ADD INDEX) are accumulated and processed when user-defined thresholds are met

The system continuously monitors and tracks all schema changes, including metrics on data ingestion (rows inserted), modification (rows updated), and deletion (rows deleted) alongside structural changes. This comprehensive approach provides users with a complete picture of both schema and data evolution.

The user interacts with the system primarily through the **Schema History interface**, which provides a comprehensive timeline of detected changes. This interface displays both the raw DDL events, data change metrics, and the analytical results generated when the pipeline is triggered.

A key innovation in our approach is the **threshold-based triggering mechanism** for non-breaking changes. The system accumulates changes until user-defined thresholds are reached, such as:

- Accumulation of a minimum number of DDL changes (e.g., 5 schema modifications)
- Data volume thresholds (e.g., 10,000 rows inserted/updated/deleted)
- Expiration of a time-based window
- Specific combination of changes affecting multiple related tables

When threshold conditions are satisfied, the analytical pipeline is automatically triggered to process the batched events, perform comparative analysis, and generate comprehensive reports.

Finally, the system provides a pro-active **alerting** mechanism that immediately notifies users of breaking changes via email, ensuring critical changes are communicated as soon as they occur.

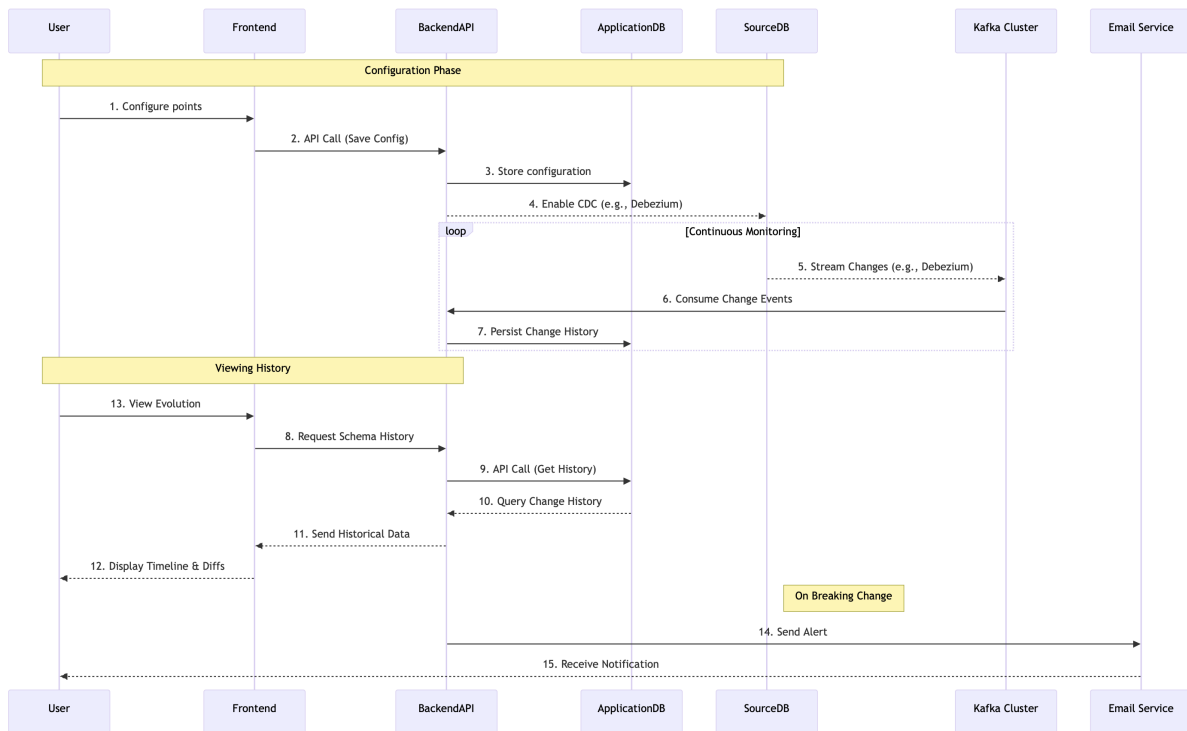


Figure 6.3: User workflow for schema change tracking and analysis

6.3.2.2 High-Level Data Flow Analysis

To understand how data moves through the system to enable the user workflow, we analyzed the high-level data flow. The process, illustrated in Figure 6.4, shows how schema and data changes in the source database propagate through our system to ultimately result in analysis, historical recording, and user notification.

The flow begins when changes occur in the source database, including both DDL statements (schema changes) and DML operations (data changes). All changes are written to the database's transaction log. Our Debezium connector, which is continuously monitoring this log, captures these events. Debezium packages each event into a structured message and publishes it to appropriate Kafka topics, ensuring durability and decoupling.

A consumer service within our application subscribes to these Kafka topics. The system implements a sophisticated routing mechanism:

- **Breaking changes** are immediately routed to real-time processing pipelines
- **Non-breaking changes** are stored in a temporary buffer for threshold evaluation

- **Data change events** (inserts, updates, deletes) are aggregated for volume metrics

The system employs a dual-mode processing architecture:

1. **Immediate processing** for breaking changes: These are processed instantly, triggering immediate alerts and updating the current schema state
2. **Threshold-based processing** for non-breaking changes: The system continuously evaluates accumulated changes against user-defined thresholds. When conditions are met (e.g., 5 DDL changes + 10,000 data operations), the analytical pipeline is triggered

When threshold conditions are satisfied, the pipeline processes the batched events comprehensively: it parses DDL statements, analyzes data change patterns, calculates metrics on ingestion and deletion volumes, performs comparative schema analysis, and persists all information to our application's database. This creates a permanent historical record that includes both structural evolution and data movement patterns.

This hybrid event-driven data flow ensures optimal resource utilization while providing both real-time responsiveness for critical changes and comprehensive batch analysis for evolutionary changes. The system maintains complete tracking of both schema modifications and data movement from the moment changes occur in the source database through to analytical processing and user notification.

6.3.3 Sprint 6 Implementation

This sprint focused on building a comprehensive change detection engine capable of handling both schema modifications (DDL) and data changes (DML). The implementation leverages a multi-stage, event-driven architecture to capture, classify, and act upon changes in real-time.

6.3.3.1 Change Data Capture (CDC) Methodology Evaluation

We evaluated two primary CDC patterns to serve as the foundation for our system, each with distinct architectural approaches.

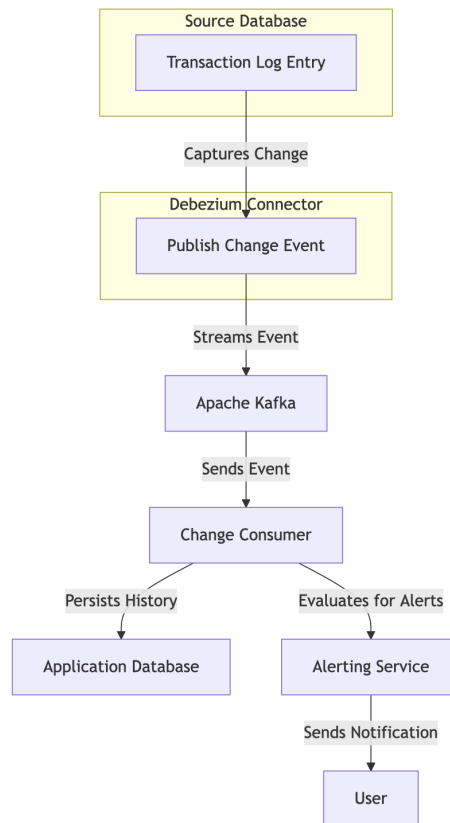


Figure 6.4: High-level data flow for hybrid schema and data change processing

1. Log-Based CDC

This method, illustrated in Figure 6.5, leverages the database’s native transaction log (e.g., MySQL’s binlog) as the source of truth. A connector reads the log entries, interprets events related to both data changes (Insert, Update, Delete) and schema modifications, and streams them to a messaging platform.

- **Strengths:**

- **Minimally Intrusive:** Requires no schema modifications; only needs read permissions on transaction logs.
- **Negligible Performance Impact:** Does not affect source database performance, as log reading is asynchronous.
- **Comprehensive Capture:** Captures all data and schema changes with high fidelity.

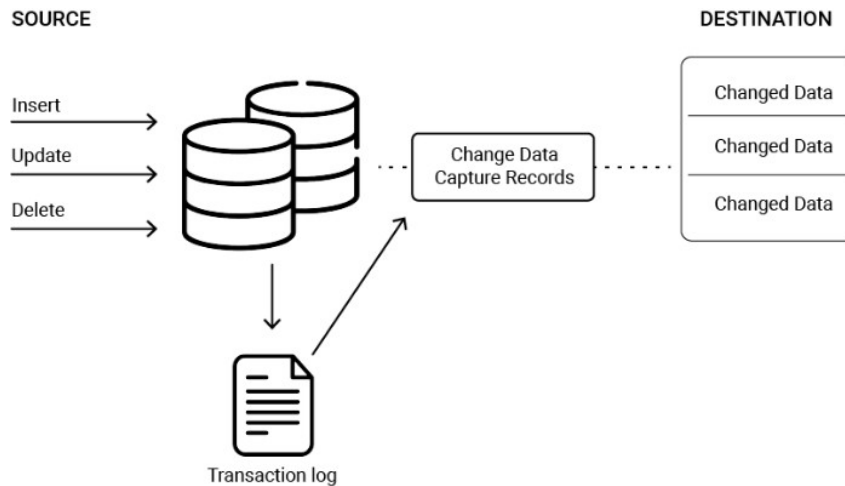


Figure 6.5: Log-Based CDC: Reading changes directly from the database transaction log.

2. Trigger-Based CDC

This method, shown in Figure 6.6, relies on creating database triggers that fire in response to data events (Insert, Update, Delete). These triggers capture the change details and write them to a dedicated "shadow" or audit table.

- **Limitations:**

- **Highly Intrusive:** Requires installing triggers within the source database, adding management overhead.
- **Performance Overhead:** Adds load to every transaction on the source database.
- **No Schema Change Capture:** This method only captures data changes (DML), not schema changes (DDL), making it insufficient for our complete use case.

6.3.3.2 Technology Selection and Architecture

Based on the evaluation, we implemented our solution using Debezium as our CDC connector and Apache Kafka as our event streaming platform. This combination provides the foundation for our event-driven architecture.

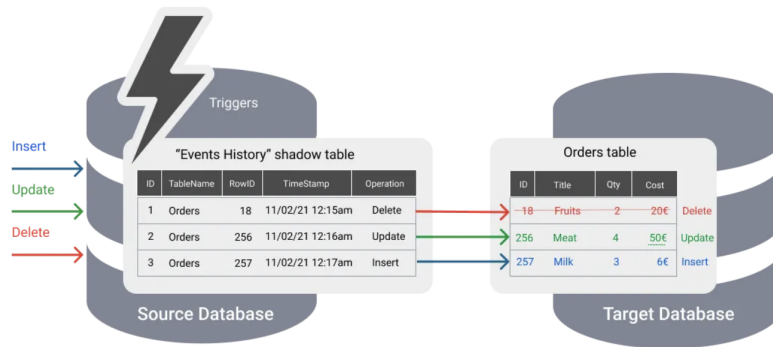


Figure 6.6: Trigger-Based CDC: Using database triggers to populate a shadow audit table.

6.3.3.3 Schema Change detection Architecture

The heart of our system is built on Debezium and Kafka, which work together like a central nervous system for data changes. Figure 6.7 shows the complete architecture overview. The

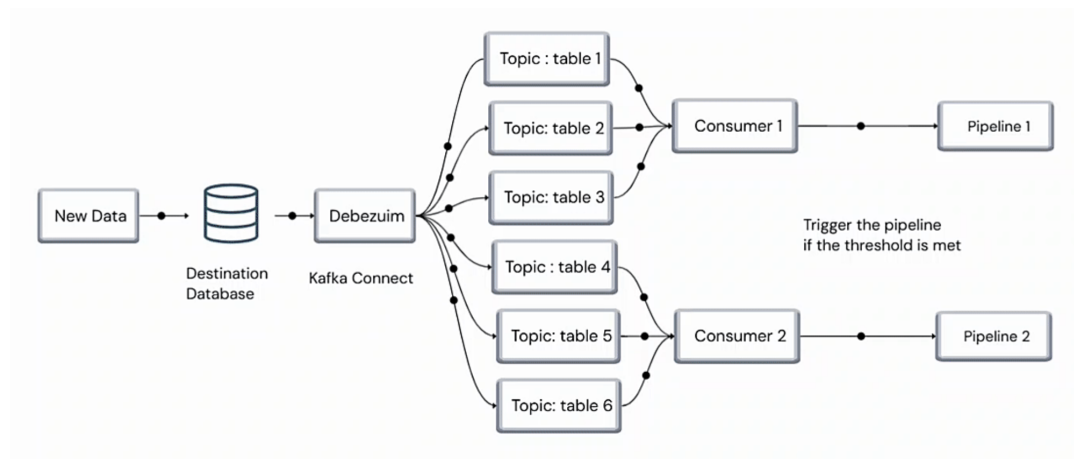


Figure 6.7: Overview of the Debezium and Kafka architecture for change capture and processing

system follows this data flow:

1. **Capture:** Debezium connectors read changes from the source database logs
2. **Stream:** Each change is published as a message to a specific Kafka topic (e.g., a topic for each database table)
3. **Process:** Different services (consumers) listen to these topics and handle specific responsibilities

This design provides several key advantages:

- **Reliability:** Kafka ensures no message is ever lost through persistence and replication

- **Scalability:** New sources or services can be added easily without affecting existing components
- **Flexibility:** Multiple applications can consume the stream of changes without affecting the database

6.3.3.4 Change Processing Engine

The Change Processing Engine is the intelligent core of our system, transforming raw schema change events into actionable insights and automated responses. Figure 6.8 illustrates this sophisticated, multi-stage decision-making process.

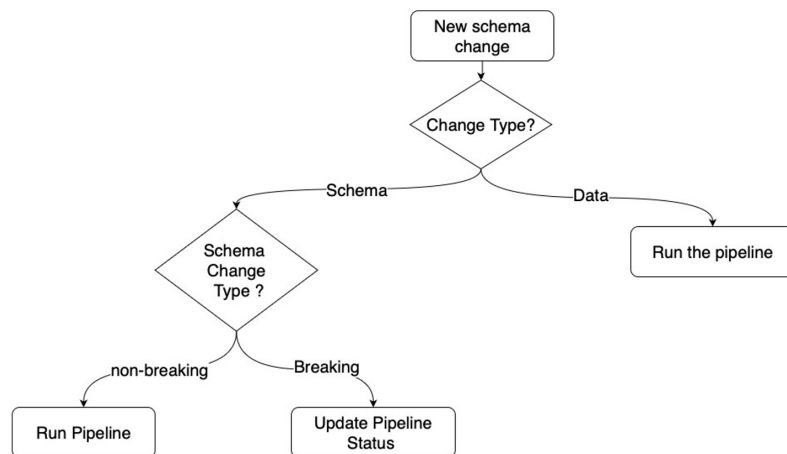


Figure 6.8: Automated workflow for processing, classifying, and reacting to schema changes

When a schema change (DDL) is detected, the system executes the following intelligent workflow:

1. Change Analysis

The engine first deconstructs and analyzes the change to understand its full context:

- **Parsing:** The raw DDL statement is parsed to extract the exact operation (e.g., 'ALTER TABLE...DROP COLUMN'), target objects (tables, columns), and modified properties.

- **Impact Assessment:** The change is cross-referenced against a registry of active data pipelines and dependent systems to identify which components are affected.

2. Change Classification

Using a rules-based classifier, the change is categorized by its potential to disrupt operations:

- **Non-Breaking Changes:** Additive or non-destructive modifications.
 - Adding a new nullable column.
 - Creating a new index on existing data.
 - Adding a table or column comment.
- **Breaking Changes:** Destructive or restrictive modifications that will cause failures in downstream pipelines or applications.
 - Dropping a table.
 - Changing a column data type (e.g., 'VARCHAR' to 'INT').
 - Adding a 'NOT NULL' constraint to an existing column.
 - Renaming a column or table.

3. Automated Response Actions

The system triggers pre-defined actions based on the classification:

For Non-Breaking Changes:

- The change is logged to the historical tracking system.
- The internal schema registry is updated to reflect the new state.
- **Threshold Analysis:** The system checks if a configurable **volume threshold** of changes has been met within a specific time window (e.g., "5+ changes in 1 hour"). If the threshold is exceeded, it can automatically trigger a data pipeline run to propagate the batch of changes efficiently, rather than processing each one individually.

- Data pipelines continue to run normally.

For Breaking Changes:

- The system immediately issues a 'pause' command to all affected data pipelines, preventing them from running and failing.
- The pipeline status is set to 'broken'.
- A high-severity alert is generated and dispatched to the engineering team.

6.3.3.5 Alerting and Notification System

The system implements a streamlined email notification protocol designed to inform all relevant stakeholders simultaneously. All database administrators and users with access permissions to the affected pipeline receive immediate email alerts when a breaking schema change is detected.

- **Comprehensive Audience:** Notifications are distributed to all authorized personnel, ensuring full visibility across teams and eliminating notification gaps.
- **Minimalist Content:** Alerts contain only the essential factual information about the change, presented clearly and without automated recommendations or analysis.
- **Immediate Delivery:** The system triggers email notifications in real-time upon detection of a breaking change, enabling rapid team response.

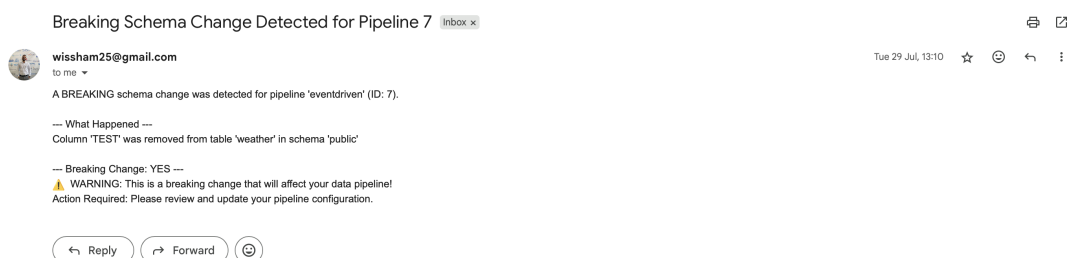


Figure 6.9: Example of a breaking change alert email.

6.4 Conclusion

Sprint 5 successfully established a secure foundation for the application by integrating Keycloak for authentication and authorization, ensuring that user access is both secure and properly governed. The implementation leveraged Keycloak's enterprise-grade features to provide robust identity management without building custom security infrastructure. We implemented comprehensive role-based access control with distinct capabilities for Admin and User roles, including sophisticated pipeline authorization that controls access based on ownership and sharing permissions.

Sprint 6 delivered the primary analytical value of the project by implementing a powerful, event-driven engine for capturing, storing, and comparing database schema changes in real-time using Debezium and Kafka. The architecture, justified by a thorough CDC benchmark, provides significant advantages over traditional polling-based approaches, including real-time detection, lower performance impact, and a complete audit trail. The implementation of a historical tracking system, a visual diff tool, and a proactive alerting mechanism provides users with a comprehensive solution for understanding and managing their database schema evolution.

The combination of these two sprints results in a secure, scalable, and functional application that is ready for users to start tracking their database schema evolution effectively and efficiently, with proper access controls ensuring data security and privacy.



GENERAL CONCLUSION

This project was created to solve a common problem in modern data operations: the heavy use of old, batch-based data systems that are slow, hard to scale, and expensive to maintain. The main idea was that a flexible, near real-time data platform could fix these issues, helping organizations become more agile and make faster, smarter decisions. The work described here has proven this idea by building a complete, ready-to-use system that connects raw data to useful business insights.

The project followed a step-by-step approach, based on solid engineering practices. The platform was designed to be modular and scalable, which allowed the main parts to be developed separately. These parts include the data ingestion module for connecting to different sources, the transformation engine for cleaning and improving data, and the visualization layer for showing results to users. One of the biggest achievements is the focus on users: the system gives business users the ability to create their own workflows and reports without depending on technical teams, which saves time and avoids delays.

Even with its success, the project still has some areas for improvement. The system works very well with structured data, but support for unstructured data can be added in the future. The monitoring and alerting system is reliable, but it could be improved with predictive analytics to detect problems before they happen. Also, while the system can scale, moving it to a container orchestration platform like Kubernetes would provide even higher availability and easier scaling for large organizations.



Webography

- [1] IVVES Consortium. (2020). *D4.1 – State of the Art on Data-Driven Engineering*. ITEA3 Project 18022. Retrieved from itea4.org
- [2] Seenivasan, D. (2022). ETL vs ELT: Choosing the right approach for your data warehouse. *International Journal for Research Trends and Innovation (IJRTI)*, 7(2). Retrieved from ResearchGate
- [3] Apache Software Foundation. (2024). *Sensors — Apache Airflow Documentation*. Retrieved from airflow.apache.org
- [4] Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Retrieved from scrumguides.org
- [5] OpenAI. (2023). *ChatGPT: Optimizing Language Models for Dialogue*. Retrieved from openai.com
- [6] n8n GmbH. (2024). *n8n - Workflow Automation Tool*. Retrieved from n8n.io
- [7] SQLFluff Contributors. (2024). *SQLFluff: The SQL Linter for Humans*. Retrieved from sqlfluff.com
- [8] Google AI. (2024). *Gemini 2.0: Next-Generation Multimodal AI Model*. Retrieved from deepmind.google
- [9] Rajkumar, N., Li, R., and Bahdanau, D. (2022). Evaluating the text-to-sql capabilities of large language models. *arXiv preprint arXiv:2204.00498*. Retrieved from arxiv.org
- [10] Starks, A. (2024). *EvaluateGPT: Comprehensive LLM Benchmarking*. GitHub repository. Retrieved from github.com

- [11] dlthub. (2024). *dlt: the data loading library for Python*. Retrieved from dlthub.com
- [12] Meltano. (2024). *Meltano Documentation*. Retrieved from docs.meltano.com
- [13] SQLMesh. (2024). *SQLMesh: Open-source scalable & efficient data transformations*. Retrieved from sqlmesh.readthedocs.io
- [14] Apache Software Foundation. (2024). *Apache Superset: Documentation*. Retrieved from superset.apache.org
- [15] Keycloak. (2024). *Keycloak Documentation*. Retrieved from keycloak.org
- [16] Debezium. (2024). *Debezium: Change Data Capture for your Databases*. Retrieved from debezium.io
- [17] Apache Software Foundation. (2024). *Apache Kafka: Documentation*. Retrieved from kafka.apache.org
- [18] Scrum.org. (2024). *Scrum Master Guide: Everything You Need to Know to Get Started*. Retrieved from scrum.org
- [19] Driessen, V. (2010). *A successful Git branching model*. Retrieved from nvie.com

A Modular Platform for Near-Real-Time Data Pipeline

Hammoudi Wissem

Résumé: Ce projet vise à concevoir et implémenter une plateforme modulaire et évolutive pour l'ingestion et la transformation de données en temps quasi réel. L'objectif principal est de surmonter les limitations des systèmes de données traditionnels, tels que la latence élevée et les coûts prohibitifs, en proposant une solution permettant aux entreprises d'exploiter pleinement leurs actifs de données pour une prise de décision rapide et éclairée. La plateforme intègre des fonctionnalités clés incluant une configuration conviviale, un traitement en temps quasi réel et un système de surveillance complet.

Mots clés: Ingestion de données, Transformation de données, Temps quasi réel, Ingénierie des données.

Abstract: This project aims to design and implement a modular and scalable platform for near-real-time data ingestion and transformation. The primary objective is to overcome the limitations of traditional data systems, such as high latency and excessive costs, by providing a solution that enables businesses to fully leverage their data assets for rapid and informed decision-making. The platform incorporates key features including user-friendly configuration, near-real-time processing, and a comprehensive monitoring system.

Keywords: Data Ingestion, Data Transformation, Near-Real-Time, Data Engineering.

ملخص: يهدف هذا المشروع إلى تصميم وتنفيذ منصة معيارية وقابلة للتطوير لاستيعاب البيانات وتحويلها في الوقت شبه الفعلي. يتمثل الهدف الرئيسي في التغلب على قيود أنظمة البيانات التقليدية، مثل البطء المفرط والتكاليف المرتفعة، من خلال تقديم حل يمكن المؤسسات من الاستفادة الكاملة من أصول بياناتها لاتخاذ قرارات سريعة ومستنيرة. تتضمن المنصة ميزات رئيسية تشمل سهولة الإعداد، والمعالجة في الوقت شبه الفعلي، ونظام مراقبة شامل.

الكلمات المفتاحية: استيعاب البيانات، تحويل البيانات، الوقت شبه الفعلي، هندسة البيانات.

A Modular Platform For Near-Real-Time Data Pipeline

Hammoudi Wissem

Abstract : This project aims to design and implement a modular and scalable platform for near-real-time data ingestion and transformation. The primary objective is to overcome the limitations of traditional data systems, such as high latency and excessive costs, by providing a solution that enables businesses to fully leverage their data assets for rapid and informed decision-making. The platform incorporates key features including user-friendly configuration, near-real-time processing, and a comprehensive monitoring system.

Key Words : Data Ingestion, Data Transformation, Near-Real-Time, Data Engineering.

Résumé : Ce projet vise à concevoir et implémenter une plateforme modulaire et évolutive pour l'ingestion et la transformation de données en temps quasi réel. L'objectif principal est de surmonter les limitations des systèmes de données traditionnels, tels que la latence élevée et les coûts prohibitifs, en proposant une solution permettant aux entreprises d'exploiter pleinement leurs actifs de données pour une prise de décision rapide et éclairée. La plateforme intègre des fonctionnalités clés incluant une configuration conviviale, un traitement en temps quasi réel et un système de surveillance complet.

Mots Clés : Ingestion de données, Transformation de données, Temps quasi réel, Ingénierie des données.

تلخيص: يهدف هذا المشروع إلى تصميم وتنفيذ منصة معيارية وقابلة للتوسع لمعالجة وإدخال البيانات في الزمن شبه الحقيقي. يتمثل الهدف الأساسي في تجاوز قيود أنظمة البيانات التقليدية، مثل البطء في الأداء وارتفاع التكاليف، وذلك عبر توفير حل يمكن المؤسسات من الاستفادة الكاملة من أصول بياناتها لاتخاذ قرارات سريعة ومدروسة. تتضمن المنصة خصائص أساسية مثل سهولة التهيئة، المعالجة شبه الفورية، ونظام مراقبة متكامل.

الكلمات المفتاحية: إدخال البيانات، تحويل البيانات، الزمن شبه الحقيقي، هندسة البيانات.